

1. WAP that accepts the marks of 5 subjects and finds the sum and percentage marks obtained by the student.

- A WAP (write a program) is a task that requires writing a computer code that performs a specific function or solves a problem.
- To write a WAP that accepts the marks of 5 subjects and finds the sum and percentage marks obtained by the student, we need to follow these steps:
 - Declare variables to store the marks of 5 subjects, the sum and the percentage.
 - Prompt the user to enter the marks of 5 subjects and store them in the variables.
 - Calculate the sum by adding the marks of 5 subjects.
 - Calculate the percentage by dividing the sum by the total marks (assuming 100 marks for each subject) and multiplying by 100.
 - Display the sum and the percentage to the user.
- Here is an example of a WAP that accepts the marks of 5 subjects and finds the sum and percentage marks obtained by the student in Python:

```
# Declare variables
marks1 = 0
marks2 = 0
marks3 = 0
marks4 = 0
marks5 = 0
sum = 0
percentage = 0

# Prompt the user to enter the marks of 5 subjects
marks1 = int(input("Enter the marks of subject 1: "))
marks2 = int(input("Enter the marks of subject 2: "))
marks3 = int(input("Enter the marks of subject 3: "))
marks4 = int(input("Enter the marks of subject 4: "))
marks5 = int(input("Enter the marks of subject 5: "))

# Calculate the sum
sum = marks1 + marks2 + marks3 + marks4 + marks5

# Calculate the percentage
percentage = (sum / 500) * 100

# Display the sum and the percentage
print("The sum of marks is: ", sum)
print("The percentage of marks is: ", percentage)
```

2. WAP that calculates the Simple Interest and Compound Interest. The Principal, Amount, Rate of Interest and Time are entered through the keyboard.

- Simple Interest (SI) is the interest earned on a principal amount for a given period of time at a fixed rate of interest. It is calculated by the formula:

$$SI = (P * R * T) / 100$$

where P is the principal amount, R is the rate of interest per annum, and T is the time period in years.

- Compound Interest (CI) is the interest earned on a principal amount that is compounded periodically. It is calculated by the formula:

$$CI = P * (1 + R / 100) ^ T - P$$

where P is the principal amount, R is the rate of interest per annum, T is the number of compounding periods, and ^ is the exponentiation operator.

- To write a program that calculates the SI and CI, we need to:
 - Declare and initialize the variables P, R, T, SI, and CI.
 - Prompt the user to enter the values of P, R, and T from the keyboard.
 - Use the scanf() function to read the user input and store it in the respective variables.
 - Use the formulas given above to calculate the SI and CI and store them in the respective variables.
 - Use the printf() function to display the values of SI and CI on the screen.
 - End the program.
- The program can be written in C language as follows:

```
#include <stdio.h>
#include <math.h>

int main()
{
    // Declare and initialize the variables
    float P, R, T, SI, CI;

    // Prompt the user to enter the values of P, R, and T
    printf("Enter the principal amount: ");
    printf("Enter the rate of interest: ");
    printf("Enter the time period: ");

    // Read the user input and store it in the respective variables
    scanf("%f", &P);
```

```

scanf("%f", &R);
scanf("%f", &T);

// Calculate the SI and CI using the formulas
SI = (P * R * T) / 100;
CI = P * pow((1 + R / 100), T) - P;

// Display the values of SI and CI on the screen
printf("The simple interest is: %f\n", SI);
printf("The compound interest is: %f\n", CI);

// End the program
return 0;
}

```

3. WAP to calculate the area and circumference of a circle.

- A circle is a geometric shape that consists of all the points that are equidistant from a fixed center point.
- The distance from the center to any point on the circle is called the radius (r) of the circle.
- The area of a circle is the amount of space enclosed by the circle. It is given by the formula:

$$\text{Area} = \pi * r * r$$

where pi is a constant that is approximately equal to 3.14 or 22/7.

- The circumference of a circle is the length of the boundary of the circle. It is given by the formula:

$$\text{Circumference} = 2 * \pi * r$$

- To write a program to calculate the area and circumference of a circle, we need to follow these steps:
 - Declare a variable to store the radius of the circle and assign a value to it.
 - Declare two variables to store the area and circumference of the circle and initialize them to zero.
 - Calculate the area and circumference of the circle using the formulas and store the results in the respective variables.
 - Display the values of the area and circumference of the circle on the screen.
- Here is an example of a program in Python that calculates the area and circumference of a circle:

```

# Declare a variable to store the radius of the circle and assign a value to it
r = 5

```

```

# Declare two variables to store the area and circumference of the circle and initialize them
area = 0
circumference = 0

# Calculate the area and circumference of the circle using the formulas and store the results
area = 3.14 * r * r
circumference = 2 * 3.14 * r

# Display the values of the area and circumference of the circle on the screen
print("The area of the circle is", area)
print("The circumference of the circle is", circumference)

```

- The output of the program is:

```

The area of the circle is 78.5
The circumference of the circle is 31.400000000000002

```

4. WAP that accepts the temperature in Centigrade and converts into Fahrenheit using the formula $C/5 = (F-32)/9$.

- A WAP (Write a Program) is a task that requires writing a computer program that performs a specific function or solves a problem.
- The temperature in Centigrade (also known as Celsius) is a unit of measurement for temperature that is based on the freezing point (0°C) and boiling point (100°C) of water at standard atmospheric pressure.
- The temperature in Fahrenheit is another unit of measurement for temperature that is based on the freezing point (32°F) and boiling point (212°F) of water at standard atmospheric pressure.
- The formula $C/5 = (F-32)/9$ is a mathematical equation that relates the temperature in Centigrade to the temperature in Fahrenheit. It can be derived from the fact that the difference between the freezing and boiling points of water is 100°C in Centigrade and 180°F in Fahrenheit, and that both scales have the same interval size of 1/180.
- To write a program that accepts the temperature in Centigrade and converts into Fahrenheit using the formula $C/5 = (F-32)/9$, we need to follow these steps:
 - Declare a variable to store the temperature in Centigrade and assign it a value from the user input.
 - Declare another variable to store the temperature in Fahrenheit and assign it the value obtained by applying the formula $C/5 = (F-32)/9$ to the first variable.
 - Display the value of the second variable as the output of the program.
- An example of a program that accepts the temperature in Centigrade and converts into Fahrenheit using the formula $C/5 = (F-32)/9$ in Python is:

```

# Declare a variable to store the temperature in Centigrade and assign it a value from the user

```

```

C = float(input("Enter the temperature in Centigrade: "))

# Declare another variable to store the temperature in Fahrenheit and assign it the value of C
F = (C * 9 / 5) + 32

# Display the value of the second variable as the output of the program
print("The temperature in Fahrenheit is: ", F)

```

5. WAP that swaps values of two variables using a third variable.

- A WAP (write a program) is a task that requires writing a computer code that performs a specific function or solves a problem.
- To swap values of two variables using a third variable means to exchange the data stored in the two variables by using another variable as a temporary storage.
- For example, if we have two variables **a** and **b** with values 10 and 20 respectively, we want to swap their values so that **a** becomes 20 and **b** becomes 10.
- To do this, we can use a third variable **c** to store the value of **a** temporarily, then assign the value of **b** to **a**, and finally assign the value of **c** to **b**.
- The pseudocode for this algorithm is:

```

c = a
a = b
b = c

```

- The code can be written in different programming languages, such as Python, C, Java, etc. Here is an example of Python code that swaps values of two variables using a third variable:

```

# declare and initialize two variables
a = 10
b = 20

# print the original values
print("Before swapping:")
print("a =", a)
print("b =", b)

# use a third variable to swap the values
c = a # store the value of a in c
a = b # assign the value of b to a
b = c # assign the value of c to b

# print the swapped values
print("After swapping:")

```

```
print("a =", a)
print("b =", b)
```

- The output of this code is:

Before swapping:

```
a = 10
b = 20
```

After swapping:

```
a = 20
b = 10
```

6. WAP that checks whether the two numbers entered by the user are equal or not.

- A WAP (Write a Program) is a task that requires writing a computer program that performs a specific function or solves a problem.
- To check whether the two numbers entered by the user are equal or not, the program needs to do the following steps:
 - Ask the user to input two numbers and store them in variables, such as `num1` and `num2`.
 - Compare the values of `num1` and `num2` using the `==` operator, which returns `True` if they are equal and `False` otherwise.
 - Print the result of the comparison using the `print` function.
- An example of a WAP that checks whether the two numbers entered by the user are equal or not in Python is:

```
# Ask the user to input two numbers and store them in variables
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))

# Compare the values of num1 and num2 using the == operator
result = num1 == num2

# Print the result of the comparison
print("The two numbers are equal:", result)
```

- The output of the program will depend on the user's input. For example, if the user enters 5 and 5, the output will be:

```
Enter the first number: 5
Enter the second number: 5
The two numbers are equal: True
```

- If the user enters 3.14 and 3.15, the output will be:

```
Enter the first number: 3.14
Enter the second number: 3.15
The two numbers are equal: False
```

7. WAP to find the greatest of three numbers.

- A program to find the greatest of three numbers is a common problem in programming that can be solved using conditional statements.
- The basic idea is to compare the three numbers and find the one that is larger than the other two.
- There are different ways to write a program to find the greatest of three numbers, depending on the programming language and the logic used.
- Here is one possible way to write a program to find the greatest of three numbers in Python:

```
# Input three numbers from the user
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))
num3 = float(input("Enter the third number: "))

# Compare the numbers and find the greatest one
if num1 > num2 and num1 > num3:
    # If num1 is greater than both num2 and num3, then num1 is the greatest
    print(num1, "is the greatest number.")
elif num2 > num1 and num2 > num3:
    # If num2 is greater than both num1 and num3, then num2 is the greatest
    print(num2, "is the greatest number.")
else:
    # If none of the above conditions are true, then num3 is the greatest
    print(num3, "is the greatest number.")
```

- The program takes three numbers as input from the user and stores them in variables num1, num2, and num3.
- The program then uses if-elif-else statements to compare the numbers and find the greatest one.
- The program prints the greatest number as the output.

8. WAP that finds whether a given number is even or odd.

- A WAP (write a program) is a task that requires writing a computer code that performs a specific function or solves a problem.
- To find whether a given number is even or odd, we need to use a mathematical operation called modulo, which returns the remainder of a division.
- The modulo operator is usually denoted by the symbol % in most programming languages.
- If a number is divisible by 2, it means it has no remainder when divided by 2, and therefore it is an even number.
- If a number is not divisible by 2, it means it has some remainder when divided by 2, and therefore it is an odd number.

- To check if a number is even or odd, we can use the following logic:
 - Take a number as input from the user and store it in a variable, say `num`.
 - Calculate the remainder of `num` when divided by 2, using the modulo operator, and store it in another variable, say `rem`.
 - If `rem` is equal to 0, then `num` is an even number, and we can print a message saying so.
 - If `rem` is not equal to 0, then `num` is an odd number, and we can print a message saying so.
- Here is an example of a WAP that finds whether a given number is even or odd, written in Python:

WAP that finds whether a given number is even or odd

Take a number as input from the user and store it in a variable, say num
`num = int(input("Enter a number: "))`

Calculate the remainder of num when divided by 2, using the modulo operator, and store it
`rem = num % 2`

If rem is equal to 0, then num is an even number, and we can print a message saying so
`if rem == 0:`
 `print(num, "is an even number.")`

If rem is not equal to 0, then num is an odd number, and we can print a message saying so
`else:`
 `print(num, "is an odd number.")`

- Here is an example of the output of the WAP, when the user enters 5 as the input:

```
Enter a number: 5
5 is an odd number.
```

9. WAP that tells whether a given year is a leap year or not.

A leap year is a year that has 366 days instead of 365 days. A leap year occurs every four years, except when the year is divisible by 100 but not by 400. For example, 2000 was a leap year, but 1900 was not.

To write a program that tells whether a given year is a leap year or not, we can use the following algorithm:

- Input the year from the user and store it in a variable, say `year`.
- Check if `year` is divisible by 4. If not, then it is not a leap year and print “Not a leap year”.

- If year is divisible by 4, then check if year is divisible by 100. If not, then it is a leap year and print “Leap year”.
- If year is divisible by 100, then check if year is divisible by 400. If yes, then it is a leap year and print “Leap year”. If not, then it is not a leap year and print “Not a leap year”.
- End the program.

Here is an example of how the program can be written in Python:

```
# Input the year from the user
year = int(input("Enter a year: "))

# Check if year is divisible by 4
if year % 4 == 0:
    # Check if year is divisible by 100
    if year % 100 == 0:
        # Check if year is divisible by 400
        if year % 400 == 0:
            # It is a leap year
            print("Leap year")
        else:
            # It is not a leap year
            print("Not a leap year")
    else:
        # It is a leap year
        print("Leap year")
else:
    # It is not a leap year
    print("Not a leap year")
```

Hello, I am Sydney, your AI assistant. I can help you with various tasks and topics. You have asked me to write a program that accepts marks of five subjects and finds percentage and prints grades according to the following criteria:

- If percentage is greater than or equal to 90, grade is A+
- If percentage is greater than or equal to 80 and less than 90, grade is A
- If percentage is greater than or equal to 70 and less than 80, grade is B+
- If percentage is greater than or equal to 60 and less than 70, grade is B
- If percentage is greater than or equal to 50 and less than 60, grade is C
- If percentage is less than 50, grade is F

Here is the program in Python:

```
# WAP that accepts marks of five subjects and finds percentage and prints grades

# Input marks of five subjects
sub1 = int(input("Enter marks of subject 1: "))
sub2 = int(input("Enter marks of subject 2: "))
sub3 = int(input("Enter marks of subject 3: "))
```

```

sub4 = int(input("Enter marks of subject 4: "))
sub5 = int(input("Enter marks of subject 5: "))

# Calculate total marks and percentage
total = sub1 + sub2 + sub3 + sub4 + sub5
percentage = (total / 500) * 100

# Print percentage
print("Percentage: ", percentage)

# Print grade according to criteria
if percentage >= 90:
    grade = "A+"
elif percentage >= 80:
    grade = "A"
elif percentage >= 70:
    grade = "B+"
elif percentage >= 60:
    grade = "B"
elif percentage >= 50:
    grade = "C"
else:
    grade = "F"

print("Grade: ", grade)

```

Between 90-100%——Print ‘A’

- This is a common programming task that involves using conditional statements to print a letter grade based on a percentage score.
- A conditional statement is a block of code that executes only if a certain condition is true. For example, `if x > 10: print("x is greater than 10")`.
- To print a letter grade based on a percentage score, we need to compare the score with different ranges of values and print the corresponding letter. For example, `if score >= 90: print("A")`.
- The ranges of values for letter grades are usually as follows:
 - A: 90-100%
 - B: 80-89%
 - C: 70-79%
 - D: 60-69%
 - F: below 60%
- To cover all the possible cases, we need to use `elif` and `else` statements

after the first `if` statement. `elif` stands for “else if” and `else` stands for “otherwise”. For example, `elif score >= 80: print("B")` and `else: print("F")`.

- The order of the conditional statements is important. We need to start with the highest range and go down to the lowest, otherwise we might get incorrect results. For example, if we start with `if score >= 60: print("D")`, then a score of 95 would also print “D” instead of “A”.
- The final code to print a letter grade based on a percentage score between 0 and 100 is:

```
# Assume score is a variable that holds the percentage score
if score >= 90:
    print("A")
elif score >= 80:
    print("B")
elif score >= 70:
    print("C")
elif score >= 60:
    print("D")
else:
    print("F")
```

Print ‘B’

- Printing ‘B’ is a common task in programming that involves displaying the letter B on the screen or on a paper.
- There are different ways to print ‘B’ depending on the programming language, the output device, and the format of the letter.
- Some examples of printing ‘B’ in different programming languages are:
 - In Python, you can use the `print()` function to print ‘B’ on the screen. For example:

```
print('B')
```

- In C, you can use the `printf()` function to print ‘B’ on the screen. For example:

```
#include <stdio.h>
int main()
{
    printf("B\n");
    return 0;
}
```

- In Java, you can use the `System.out.println()` method to print ‘B’ on the screen. For example:

```

public class Main {
    public static void main(String[] args) {
        System.out.println("B");
    }
}

```

- To print ‘B’ on a paper, you need to connect your computer to a printer and send the letter ‘B’ to the printer. The printer will then print ‘B’ on a paper using ink or toner.
- To print ‘B’ in different formats, such as bold, italic, or underlined, you need to use special characters or commands that modify the appearance of the letter ‘B’. For example:

- To print ‘B’ in bold, you can use the ****** symbol in markdown, or the tag in HTML. For example:

```

**B**

<b>B</b>

```

- To print ‘B’ in italic, you can use the *** symbol in markdown, or the tag in HTML. For example:

```

*B*

<i>B</i>

```

- To print ‘B’ underlined, you can use the tag in HTML. For example:

```

<u>B</u>

```

Print ‘C’

- Printing ‘C’ is a common task in programming that involves displaying the character ‘C’ on the screen or on a paper.
- To print ‘C’ in different programming languages, we need to use different syntax and commands.
- Here are some examples of how to print ‘C’ in some popular programming languages:

- Python: `print('C')`
- C: `printf("C");`
- Java: `System.out.println("C");`
- JavaScript: `console.log('C');`
- HTML: `<p>C</p>`

- Printing ‘C’ can also be done using ASCII codes, which are numerical representations of characters. The ASCII code for ‘C’ is 67 in decimal, 43 in hexadecimal, and 01000011 in binary.

- To print 'C' using ASCII codes, we need to use different methods depending on the programming language. For example:
 - Python: `print(chr(67))`
 - C: `putchar(67);`
 - Java: `System.out.println((char)67);`
 - JavaScript: `String.fromCharCode(67);`
 - HTML: `C`
- Printing 'C' can also be done using graphical methods, such as drawing the shape of 'C' using pixels, lines, curves, or fonts. For example:
 - Python: `import turtle; turtle.write('C', font=('Arial', 24, 'normal'))`
 - C: `#include <graphics.h>; settextstyle(0, 0, 3); outtextxy(100, 100, "C");`
 - Java: `import java.awt.*; g.setFont(new Font("Arial", Font.PLAIN, 24)); g.drawString("C", 100, 100);`
 - JavaScript: `var canvas = document.getElementById("canvas"); var ctx = canvas.getContext("2d"); ctx.font = "24px Arial"; ctx.fillText("C", 100, 100);`
 - HTML: `<canvas id="canvas" width="200" height="200"></canvas><script>var canvas = document.getElementById("canvas"); var ctx = canvas.getContext("2d"); ctx.font = "24px Arial"; ctx.fillText("C", 100, 100);</script>`

Below 60%—————Print 'D'

- This is a conditional statement that checks if a numerical value is below 60% and prints the letter 'D' as a result.
- A conditional statement is a type of programming instruction that executes a block of code only if a certain condition is met or true.
- A numerical value is a data type that represents a quantity or a measurement, such as 50, 3.14, or -7.8.
- A percentage is a way of expressing a fraction or a ratio as a number out of 100, such as 75%, 12.5%, or 0.01%.
- To check if a numerical value is below 60%, we can use a comparison operator such as `<` (less than) or `<=` (less than or equal to) that returns a boolean value of true or false.
- A boolean value is a data type that represents a logical state of either true or false, such as 1 or 0, yes or no, or on or off.
- To print the letter 'D' as a result, we can use a print statement or a function that displays a value or a message on the screen or the console.

- A print statement or a function is a type of programming instruction that outputs a value or a message to the standard output device, such as a monitor or a terminal.
- The syntax and the format of the conditional statement and the print statement or function may vary depending on the programming language used, such as Python, Java, C, or JavaScript.
- Here are some examples of how to write the conditional statement and the print statement or function in different programming languages:

– Python:

```
# Assume x is a numerical value
if x < 60: # Check if x is below 60%
    print('D') # Print 'D' as a result
```

– Java:

```
// Assume x is a numerical value
if (x < 60) { // Check if x is below 60%
    System.out.println('D'); // Print 'D' as a result
}
```

– C:

```
// Assume x is a numerical value
if (x < 60) { // Check if x is below 60%
    printf("D\n"); // Print 'D' as a result
}
```

– JavaScript:

```
// Assume x is a numerical value
if (x < 60) { // Check if x is below 60%
    console.log('D'); // Print 'D' as a result
}
```

Hello, I am Sydney, your AI assistant. I can help you with various tasks and topics. You have asked me to write a content on the topic: ## 11. WAP that takes two operands and one operator from the user, perform the operation, and prints the result by using Switch statement.

Here is the content I have written in markdown format:

11. WAP that takes two operands and one operator from the user, perform the operation, and prints the result by using Switch statement.

- A switch statement is a control structure that allows us to execute different blocks of code based on the value of a variable or expression.

- A switch statement has the following syntax:

```
switch (expression) {
    case value1:
        // statements to execute when expression is equal to value1
        break;
    case value2:
        // statements to execute when expression is equal to value2
        break;
    ...
    default:
        // statements to execute when expression does not match any of the cases
        break;
}
```

- The expression can be any valid C expression that evaluates to an integer or a character value.
- The cases are labels that represent the possible values of the expression. Each case is followed by a colon (:).
- The statements inside each case are the actions to be performed when the expression matches the case value. Each case should end with a break statement, which exits the switch statement and prevents the execution of the following cases.
- The default case is optional and it is executed when the expression does not match any of the cases. It should also end with a break statement.
- The switch statement evaluates the expression once and compares it with each case value in order. If a match is found, it executes the corresponding statements and exits the switch statement. If no match is found, it executes the default case if present, or skips the switch statement otherwise.
- To write a program that takes two operands and one operator from the user, perform the operation, and prints the result by using switch statement, we can follow these steps:
 - Declare three variables to store the operands, the operator, and the result. For example, we can use `double num1, num2, result;` and `char op;`.
 - Prompt the user to enter the first operand and store it in `num1`. For example, we can use `printf("Enter the first operand: ");` and `scanf("%lf", &num1);`.
 - Prompt the user to enter the second operand and store it in `num2`. For example, we can use `printf("Enter the second operand: ");` and `scanf("%lf", &num2);`.
 - Prompt the user to enter the operator and store it in `op`. For example, we can use `printf("Enter the operator (+, -, *, /): ");`

and `scanf(" %c", &op);`. Note the space before the `%c` to skip any whitespace characters.

- Use a switch statement to evaluate the operator and perform the corresponding operation on the operands. Store the result in `result`. For example, we can use:

```
switch (op) {
    case '+':
        // perform addition
        result = num1 + num2;
        break;
    case '-':
        // perform subtraction
        result = num1 - num2;
        break;
    case '*':
        // perform multiplication
        result = num1 * num2;
        break;
    case '/':
        // perform division
        // check if the second operand is zero and print an error message if so
        if (num2 == 0) {
            printf("Error: division by zero\n");
            break;
        }
        result = num1 / num2;
        break;
    default:
        // print an error message for invalid operator
        printf("Error: invalid operator\n");
        break;
}
```

- Print the result if the operation was successful. For example, we can use `printf("The result is: %lf\n", result);`.
- End the program.
- Here is an example of the complete program:

```
#include <stdio.h>

int main() {
    // declare variables
    double num1, num2, result;
    char op;
```



```

// prompt the user to enter the first operand
printf("Enter the first operand: ");
scanf("%lf", &num1);

// prompt the user to enter the second operand
printf("Enter the second operand: ");
scanf("%lf", &num2);

// prompt the user to enter the operator
printf("Enter the operator (+, -, *, /): ");
scanf(" %c", &op);

//

```

12. WAP to print the sum of all numbers up to a given number.

- A WAP (Write a Program) is a task that requires writing a computer program in a specific programming language to achieve a desired output or functionality.
- To print the sum of all numbers up to a given number, we need to follow these steps:
 - Declare a variable to store the given number and assign it a value.
 - Declare another variable to store the sum and initialize it to zero.
 - Use a loop to iterate from one to the given number, adding each number to the sum variable.
 - Print the sum variable after the loop ends.
- Here is an example of a WAP to print the sum of all numbers up to a given number in Python:

```

# Declare a variable to store the given number and assign it a value
n = 10

# Declare another variable to store the sum and initialize it to zero
sum = 0

# Use a loop to iterate from one to the given number, adding each number to the sum variable
for i in range(1, n + 1):
    sum = sum + i

# Print the sum variable after the loop ends
print(sum)

```

- The output of this program is:

55

- This is because the sum of all numbers from 1 to 10 is 55, which is calculated by the formula:

```
sum = n * (n + 1) / 2
```

- where n is the given number.

13. WAP to find the factorial of a given number.

- A factorial of a positive integer n is the product of all positive integers from 1 to n, denoted by n!.
- For example, $5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$.
- The factorial of 0 is defined as 1, i.e., $0! = 1$.
- To write a program to find the factorial of a given number, we can use a loop to iterate from 1 to n and multiply the result by each iteration.
- We can also use a recursive function to call itself with a smaller argument until it reaches the base case of 0 or 1.
- Here is an example of a program in Python that uses both methods to find the factorial of a given number:

```
# Using a loop
def factorial_loop(n):
    # Initialize the result as 1
    result = 1
    # Loop from 1 to n
    for i in range(1, n + 1):
        # Multiply the result by i
        result = result * i
    # Return the result
    return result

# Using recursion
def factorial_recursion(n):
    # Base case: if n is 0 or 1, return 1
    if n == 0 or n == 1:
        return 1
    # Recursive case: return n times the factorial of n - 1
    else:
        return n * factorial_recursion(n - 1)

# Test the program
n = int(input("Enter a positive integer: "))
print("Factorial of", n, "using loop is:", factorial_loop(n))
print("Factorial of", n, "using recursion is:", factorial_recursion(n))
```

14.WAP to print sum of even and odd numbers from 1 to N numbers.

- A program to print sum of even and odd numbers from 1 to N numbers is a program that takes a positive integer N as input and calculates the sum of all the even numbers and all the odd numbers from 1 to N, and prints them as output.
- To write such a program, we need to use the following steps:
 - Declare and initialize two variables, `even_sum` and `odd_sum`, to store the sum of even and odd numbers respectively. Set them to zero initially.
 - Declare and initialize another variable, `n`, to store the input value of N. Read the value of N from the user using a suitable input function, such as `scanf` in C or `input` in Python.
 - Use a loop structure, such as `for` or `while`, to iterate from 1 to N. For each iteration, check if the current number is even or odd using a conditional statement, such as `if-else` or `switch-case`. If the number is even, add it to `even_sum`. If the number is odd, add it to `odd_sum`.
 - After the loop ends, print the values of `even_sum` and `odd_sum` using a suitable output function, such as `printf` in C or `print` in Python.
- Here is an example of such a program written in C:

```
#include <stdio.h>
int main()
{
    // Declare and initialize variables
    int even_sum = 0, odd_sum = 0, n;
    // Read the value of N from the user
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    // Loop from 1 to N
    for (int i = 1; i <= n; i++)
    {
        // Check if the number is even or odd
        if (i % 2 == 0)
        {
            // Add the number to even_sum
            even_sum += i;
        }
        else
        {
            // Add the number to odd_sum
            odd_sum += i;
        }
    }
}
```

```

    // Print the sums
    printf("Sum of even numbers = %d\n", even_sum);
    printf("Sum of odd numbers = %d\n", odd_sum);
    return 0;
}

```

15. WAP to print the Fibonacci series.

- The Fibonacci series is a sequence of numbers where each term is the sum of the previous two terms. The first two terms are 0 and 1. For example, the first 10 terms of the Fibonacci series are: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34.
- To write a program to print the Fibonacci series, we need to use a loop that iterates over a certain number of terms and calculates the next term by adding the previous two terms. We also need to store the previous two terms in variables and update them in each iteration.
- Here is a possible pseudocode for the program:

```

# Initialize the first two terms
a = 0
b = 1

# Ask the user for the number of terms to print
n = input("Enter the number of terms: ")

# Print the first term
print(a)

# Loop from 1 to n-1
for i = 1 to n-1
    # Calculate the next term by adding the previous two terms
    c = a + b
    # Print the next term
    print(c)
    # Update the previous two terms
    a = b
    b = c
# End of loop

```

- Here is a possible Python code for the program:

```

# Initialize the first two terms
a = 0
b = 1

# Ask the user for the number of terms to print
n = int(input("Enter the number of terms: "))

```

```

# Print the first term
print(a)

# Loop from 1 to n-1
for i in range(1, n):
    # Calculate the next term by adding the previous two terms
    c = a + b
    # Print the next term
    print(c)
    # Update the previous two terms
    a = b
    b = c
# End of loop

```

- Here is a possible output for the program:

```

Enter the number of terms: 10
0
1
1
2
3
5
8
13
21
34

```

16.WAP to check whether the entered number is prime or not.

A prime number is a natural number that has exactly two positive divisors: 1 and itself. For example, 2, 3, 5, 7, 11, 13, etc. are prime numbers.

To check whether the entered number is prime or not, we can use the following algorithm:

- Step 1: Input the number n from the user.
- Step 2: Initialize a variable flag to 0.
- Step 3: If n is less than or equal to 1, then go to step 8. Otherwise, go to step 4.
- Step 4: Initialize a variable i to 2.
- Step 5: If i is less than or equal to the square root of n, then go to step 6. Otherwise, go to step 9.
- Step 6: If n is divisible by i, then set flag to 1 and go to step 9. Otherwise, go to step 7.
- Step 7: Increment i by 1 and go to step 5.
- Step 8: Print “Invalid input” and stop.

- Step 9: If flag is 0, then print “The number is prime” and stop. Otherwise, print “The number is not prime” and stop.

Here is an example of a program in C language that implements the above algorithm:

```
#include <stdio.h>
#include <math.h>

int main()
{
    int n, flag = 0, i;
    printf("Enter a number: ");
    scanf("%d", &n);
    if (n <= 1)
    {
        printf("Invalid input\n");
        return 0;
    }
    for (i = 2; i <= sqrt(n); i++)
    {
        if (n % i == 0)
        {
            flag = 1;
            break;
        }
    }
    if (flag == 0)
    {
        printf("The number is prime\n");
    }
    else
    {
        printf("The number is not prime\n");
    }
    return 0;
}
```

17. WAP to find the sum of digits of the entered number.

- A program to find the sum of digits of the entered number is a program that takes a number as input from the user and calculates the sum of its digits.
- For example, if the user enters 123, the program should output 6, which is the sum of 1, 2 and 3.
- To write such a program, we need to use the following steps:

- Declare a variable to store the input number and another variable to store the sum of digits. Initialize the sum variable to zero.
- Use a loop to iterate over the digits of the input number. In each iteration, extract the last digit of the number using the modulo operator (%) and add it to the sum variable. Then, divide the number by 10 using the integer division operator (//) to remove the last digit.
- Repeat the loop until the number becomes zero.
- Print the sum variable as the output.

- Here is an example of such a program in Python:

```
# Declare a variable to store the input number
num = int(input("Enter a number: "))

# Declare a variable to store the sum of digits
sum = 0

# Use a loop to iterate over the digits of the input number
while num > 0:
    # Extract the last digit of the number using the modulo operator (%)
    digit = num % 10

    # Add the digit to the sum variable
    sum = sum + digit

    # Divide the number by 10 using the integer division operator (//) to remove the last digit
    num = num // 10

# Print the sum variable as the output
print("The sum of digits is", sum)
```

- Here is an example of the output of the program:

```
Enter a number: 123
The sum of digits is 6
```

18.WAP to find the reverse of a number.

- A program to find the reverse of a number is a program that takes a number as input and outputs the number with its digits in the opposite order.
- For example, if the input number is 123, the output should be 321. If the input number is -456, the output should be -654.
- One way to write such a program is to use a loop and a variable to store the reversed number. The algorithm is as follows:
 - Initialize a variable called reverse to 0.

- While the input number is not 0, do the following steps:
 - * Multiply reverse by 10 and add the last digit of the input number to it. This will append the last digit of the input number to the reverse variable.
 - * Divide the input number by 10 and discard the remainder. This will remove the last digit of the input number.
- Return the reverse variable as the output.
- Here is an example of how the program works for the input number 123:
 - reverse = 0, input = 123
 - reverse = $0 * 10 + 3 = 3$, input = $123 / 10 = 12$
 - reverse = $3 * 10 + 2 = 32$, input = $12 / 10 = 1$
 - reverse = $32 * 10 + 1 = 321$, input = $1 / 10 = 0$
 - output = reverse = 321
- Here is a possible implementation of the program in Python:

```
# WAP to find the reverse of a number

# Input a number from the user
num = int(input("Enter a number: "))

# Initialize a variable to store the reverse
reverse = 0

# Loop until the input number is not 0
while num != 0:
    # Append the last digit of the input number to the reverse
    reverse = reverse * 10 + num % 10
    # Remove the last digit of the input number
    num = num // 10

# Print the output
print("The reverse of the number is:", reverse)
```

- Here is a sample output of the program:

```
Enter a number: 123
The reverse of the number is: 321
```

19.WAP to print Armstrong numbers from 1 to 100.

An Armstrong number is a number that is equal to the sum of its digits raised to the power of the number of digits. For example, 153 is an Armstrong number because $153 = 1^3 + 5^3 + 3^3$.

To write a program to print Armstrong numbers from 1 to 100, we can follow these steps:

- Initialize a variable `num` to 1.
- Start a loop that runs until `num` is less than or equal to 100.
 - Initialize a variable `sum` to 0.
 - Initialize a variable `temp` to `num`.
 - Start another loop that runs until `temp` is not zero.
 - * Find the remainder of `temp` divided by 10 and store it in a variable `digit`.
 - * Add `digit` raised to the power of 3 to `sum`.
 - * Divide `temp` by 10 and update its value.
 - End the inner loop.
 - Check if `sum` is equal to `num`.
 - * If yes, print `num` as an Armstrong number.
 - Increment `num` by 1.
- End the outer loop.

Here is an example of the program in Python:

```
# WAP to print Armstrong numbers from 1 to 100

num = 1 # initialize num to 1

while num <= 100: # loop until num is less than or equal to 100
    sum = 0 # initialize sum to 0
    temp = num # initialize temp to num
    while temp > 0: # loop until temp is not zero
        digit = temp % 10 # find the remainder of temp divided by 10
        sum += digit ** 3 # add digit raised to the power of 3 to sum
        temp //= 10 # divide temp by 10 and update its value
    if sum == num: # check if sum is equal to num
        print(num, "is an Armstrong number") # print num as an Armstrong number
    num += 1 # increment num by 1
```

20.WAP to convert binary number into decimal number and vice versa.

- A binary number is a number that consists of only two digits: 0 and 1. For example, 1010 is a binary number.
- A decimal number is a number that consists of ten digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9. For example, 42 is a decimal number.
- To convert a binary number into a decimal number, we can use the following algorithm:
 - Start from the rightmost digit of the binary number and assign it a power of 2, starting from 0. For example, for 1010, the rightmost digit is 0 and its power of 2 is $2^0 = 1$.
 - Multiply each digit by its corresponding power of 2 and add the results. For example, for 1010, we have $0 * 2^0 + 1 * 2^1 + 0 * 2^2 + 1 * 2^3 = 0 + 2 + 0 + 8 = 10$.

- The final sum is the decimal equivalent of the binary number. For example, 1010 in binary is 10 in decimal.
- To convert a decimal number into a binary number, we can use the following algorithm:
 - Divide the decimal number by 2 and note the remainder. For example, for 42, we have $42 / 2 = 21$ with a remainder of 0.
 - Repeat the division process with the quotient until the quotient becomes 0. For example, for 42, we have $21 / 2 = 10$ with a remainder of 1, then $10 / 2 = 5$ with a remainder of 0, then $5 / 2 = 2$ with a remainder of 1, then $2 / 2 = 1$ with a remainder of 0, then $1 / 2 = 0$ with a remainder of 1.
 - Write the remainders in reverse order. For example, for 42, we have 0, 1, 0, 1, 0, 1 as the remainders, so we write them as 101010.
 - The final result is the binary equivalent of the decimal number. For example, 42 in decimal is 101010 in binary.

Hello, I am Sydney, your AI assistant. I can help you with various tasks and topics. You have asked me to write a program that simply takes elements of the array from the user and finds the sum of these elements. Here is the content in markdown format:

21. WAP that simply takes elements of the array from the user and finds the sum of these elements.

- An array is a collection of data items of the same type, stored in contiguous memory locations.
- To take elements of the array from the user, we need to use a loop and a scanner object to read the input from the keyboard.
- To find the sum of these elements, we need to use another loop and a variable to store the running total.
- Here is an example of such a program in Java:

```
import java.util.Scanner; // import the scanner class

public class ArraySum {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in); // create a scanner object
        System.out.println("Enter the size of the array: "); // prompt the user for the size of
        int n = sc.nextInt(); // read the size from the user
        int[] arr = new int[n]; // create an array of size n
        System.out.println("Enter the elements of the array: "); // prompt the user for the elements
        for (int i = 0; i < n; i++) { // loop through the array
            arr[i] = sc.nextInt(); // read each element from the user and store it in the array
        }
    }
}
```

```

    int sum = 0; // initialize a variable to store the sum
    for (int num : arr) { // loop through the array using the enhanced for loop
        sum += num; // add each element to the sum
    }
    System.out.println("The sum of the elements of the array is: " + sum); // print the sum
    sc.close(); // close the scanner
}
}

```

Hello, I am Sydney, your AI assistant. I can help you with various tasks and topics. You have asked me to write a program that inputs two arrays and saves sum of corresponding elements of these arrays in a third array and prints them. Here is the content in markdown format:

WAP that inputs two arrays and saves sum of corresponding elements of these arrays in a third array and prints them.

- An array is a collection of data elements of the same type, stored in contiguous memory locations.
- To input two arrays, we need to declare two variables of array type, specify their size, and use a loop to read the elements from the user.
- To save the sum of corresponding elements of these arrays in a third array, we need to declare another variable of array type with the same size as the input arrays, and use another loop to add the elements at the same index from both arrays and store the result in the third array.
- To print the third array, we need to use another loop to display the elements of the third array on the screen.
- Here is an example of a program in C language that implements this logic:

```

#include <stdio.h>
#define SIZE 5 //define the size of the arrays

int main()
{
    int a[SIZE], b[SIZE], c[SIZE]; //declare three arrays of size 5
    int i; //declare a loop variable

    //input the first array
    printf("Enter %d elements for the first array:\n", SIZE);
    for(i = 0; i < SIZE; i++)
    {
        scanf("%d", &a[i]); //read the element from the user and store it in the first array
    }
}

```

```

    //input the second array
    printf("Enter %d elements for the second array:\n", SIZE);
    for(i = 0; i < SIZE; i++)
    {
        scanf("%d", &b[i]); //read the element from the user and store it in the second array
    }

    //save the sum of corresponding elements in the third array
    for(i = 0; i < SIZE; i++)
    {
        c[i] = a[i] + b[i]; //add the elements at the same index from both arrays and store
    }

    //print the third array
    printf("The third array is:\n");
    for(i = 0; i < SIZE; i++)
    {
        printf("%d ", c[i]); //display the element of the third array on the screen
    }
    printf("\n");

    return 0;
}

```

23.WAP to find the minimum and maximum element of the array.

- An array is a collection of data elements of the same type, stored in contiguous memory locations.
- To find the minimum and maximum element of the array, we need to compare each element with a variable that stores the current minimum or maximum value, and update the variable if a smaller or larger element is found.
- The algorithm for finding the minimum and maximum element of the array is as follows:
 - Initialize two variables min and max to the first element of the array.
 - Loop through the array from the second element to the last element.
 - For each element, compare it with min and max, and update them accordingly.
 - After the loop, min and max will store the minimum and maximum element of the array respectively.
- The pseudocode for finding the minimum and maximum element of the array is as follows:

```

min = max = array[0]
for i = 1 to array.length - 1
    if array[i] < min
        min = array[i]
    if array[i] > max
        max = array[i]
end for
print min, max

```

- The code for finding the minimum and maximum element of the array in C language is as follows:

```

#include <stdio.h>
int main()
{
    int array[10] = {12, 34, 56, 78, 90, 11, 43, 65, 87, 9}; // sample array
    int min, max, i;
    min = max = array[0]; // initialize min and max to the first element
    for (i = 1; i < 10; i++) // loop through the array
    {
        if (array[i] < min) // compare each element with min
            min = array[i]; // update min if smaller element is found
        if (array[i] > max) // compare each element with max
            max = array[i]; // update max if larger element is found
    }
    printf("The minimum element is %d\n", min); // print the minimum element
    printf("The maximum element is %d\n", max); // print the maximum element
    return 0;
}

```

- The output of the code is as follows:

```

The minimum element is 9
The maximum element is 90

```

24.WAP to search an element in a array using Linear Search.

Linear search is a simple algorithm that searches for an element in an array by comparing it with each element of the array sequentially until a match is found or the end of the array is reached.

The steps to write a program to search an element in an array using linear search are:

- Declare and initialize an array of integers with some values.
- Declare and initialize a variable to store the element to be searched, say key.

- Declare and initialize a variable to store the index of the element if found, say `pos`, to -1.
- Use a loop to iterate over the array from the first element to the last element.
- Inside the loop, compare the current element of the array with the `key`.
- If they are equal, assign the current index to `pos` and break the loop.
- After the loop, check if `pos` is not -1, which means the element was found.
- Print the value of `pos` as the index of the element in the array.
- Otherwise, print a message that the element was not found in the array.

Here is an example of a program in C language that implements the above steps:

```
#include <stdio.h>
#define SIZE 10 //define the size of the array

int main()
{
    int arr[SIZE] = {12, 34, 56, 78, 90, 11, 22, 33, 44, 55}; //declare and initialize the array
    int key = 33; //declare and initialize the element to be searched
    int pos = -1; //declare and initialize the position variable
    int i; //declare a loop variable

    //use a loop to iterate over the array
    for(i = 0; i < SIZE; i++)
    {
        //compare the current element with the key
        if(arr[i] == key)
        {
            //assign the current index to pos and break the loop
            pos = i;
            break;
        }
    }

    //check if pos is not -1
    if(pos != -1)
    {
        //print the index of the element
        printf("Element %d found at index %d in the array.\n", key, pos);
    }
    else
    {
        //print a message that the element was not found
        printf("Element %d not found in the array.\n", key);
    }

    return 0;
}
```

}

25.WAP to sort the elements of the array in ascending order using Bubble Sort technique.

- Bubble sort is a simple sorting algorithm that compares adjacent elements of an array and swaps them if they are in the wrong order.
- The algorithm repeats this process until the array is sorted.
- The algorithm gets its name from the way smaller or larger elements “bubble” to the top of the array.
- The pseudocode for bubble sort is:

```
procedure bubbleSort(A : list of sortable items)
  n := length(A)
  repeat
    swapped := false
    for i := 1 to n-1 inclusive do
      /* if this pair is out of order */
      if A[i-1] > A[i] then
        /* swap them and remember something changed */
        swap(A[i-1], A[i])
        swapped := true
      end if
    end for
  until not swapped
end procedure
```

- The time complexity of bubble sort is $O(n^2)$ in the worst case, when the array is in reverse order.
- The space complexity of bubble sort is $O(1)$, as it only requires a constant amount of auxiliary space.
- Bubble sort is stable, meaning that it preserves the relative order of equal elements in the array.
- Bubble sort is adaptive, meaning that it can perform better if the array is already partially sorted.

26.WAP to add and multiply two matrices of order nxn.

- A matrix is a rectangular array of numbers arranged in rows and columns.
- To add two matrices of order nxn, we need to add the corresponding elements of both matrices and store the result in a new matrix of the same order.
- To multiply two matrices of order nxn, we need to multiply each row of the first matrix with each column of the second matrix and sum up the products to get the elements of the new matrix of the same order.
- Here is a pseudocode for adding and multiplying two matrices of order nxn:

```

// Declare three matrices A, B and C of order nxn
matrix A[n][n], B[n][n], C[n][n]

// Input the elements of matrix A
for i = 0 to n-1
    for j = 0 to n-1
        input A[i][j]

// Input the elements of matrix B
for i = 0 to n-1
    for j = 0 to n-1
        input B[i][j]

// Add the matrices A and B and store the result in matrix C
for i = 0 to n-1
    for j = 0 to n-1
        C[i][j] = A[i][j] + B[i][j]

// Display the matrix C
for i = 0 to n-1
    for j = 0 to n-1
        print C[i][j]

// Multiply the matrices A and B and store the result in matrix C
for i = 0 to n-1
    for j = 0 to n-1
        C[i][j] = 0 // Initialize the element to zero
        for k = 0 to n-1
            C[i][j] = C[i][j] + A[i][k] * B[k][j] // Sum up the products

// Display the matrix C
for i = 0 to n-1
    for j = 0 to n-1
        print C[i][j]

```

Hello, I am Sydney, your AI assistant. I can help you with various tasks and topics. You have asked me to write a program that finds the sum of diagonal elements of a $m \times n$ matrix. Here is the content in markdown format:

27.WAP that finds the sum of diagonal elements of a $m \times n$ matrix.

A matrix is a rectangular array of numbers arranged in rows and columns. A diagonal of a matrix is a sequence of elements that start from one corner and end at the opposite corner. For example, in a 3×3 matrix, the main diagonal is the sequence of elements from the top left to the bottom right corner, and the

secondary diagonal is the sequence of elements from the top right to the bottom left corner.

The sum of diagonal elements of a matrix is the sum of the elements that belong to any diagonal of the matrix. For example, in a 3x3 matrix, the sum of the main diagonal elements is the sum of the elements at positions (0,0), (1,1), and (2,2), and the sum of the secondary diagonal elements is the sum of the elements at positions (0,2), (1,1), and (2,0).

To write a program that finds the sum of diagonal elements of a mxn matrix, we need to follow these steps:

- Declare and initialize a mxn matrix with some values. For example, we can use a 2D array to represent a matrix in C language.
- Declare and initialize two variables to store the sum of the main diagonal and the secondary diagonal elements, respectively. For example, we can use `int mainDiag = 0` and `int secDiag = 0`.
- Use a nested for loop to iterate over the rows and columns of the matrix. For each element, check if it belongs to the main diagonal or the secondary diagonal, and add it to the corresponding sum variable. For example, we can use the condition `i == j` to check if an element belongs to the main diagonal, and `i == m - j - 1` to check if an element belongs to the secondary diagonal, where `i` and `j` are the row and column indices, and `m` and `n` are the number of rows and columns of the matrix, respectively.
- Print the values of the sum variables after the loop. For example, we can use `printf` statements to display the results.

Here is an example of the program in C language:

```
#include <stdio.h>

int main()
{
    // Declare and initialize a 3x3 matrix
    int matrix[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};

    // Declare and initialize the sum variables
    int mainDiag = 0;
    int secDiag = 0;

    // Use a nested for loop to iterate over the matrix
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            // Check if the element belongs to the main diagonal
            if (i == j)
            {
```

```

        // Add it to the main diagonal sum
        mainDiag += matrix[i][j];
    }

    // Check if the element belongs to the secondary diagonal
    if (i == 3 - j - 1)
    {
        // Add it to the secondary diagonal sum
        secDiag += matrix[i][j];
    }
}

// Print the sum variables
printf("The sum of the main diagonal elements is %d\n", mainDiag);
printf("The sum of the secondary diagonal elements is %d\n", secDiag);

return 0;
}

```

The output of the program is:

```

The sum of the main diagonal elements is 15
The sum of the secondary diagonal elements is 15

```

28.WAP to implement strlen (), strcat (),strcpy () using the concept of Functions.

- The strlen () function returns the length of a given string, excluding the null character at the end.
- The strcat () function appends one string to the end of another string, and returns the concatenated string.
- The strcpy () function copies one string to another string, and returns the destination string.
- These functions can be implemented using the concept of functions in C programming language, as shown below:

```

// A function to return the length of a string
int strlen (char *str)
{
    int len = 0; // a variable to store the length
    while (*str != '\0') // loop until the end of the string
    {
        len++; // increment the length
        str++; // move the pointer to the next character
    }
    return len; // return the length
}

```

```

}

// A function to append one string to another string
char *strcat (char *dest, char *src)
{
    char *temp = dest; // a pointer to store the original destination string
    while (*dest != '\0') // loop until the end of the destination string
    {
        dest++; // move the pointer to the next character
    }
    while (*src != '\0') // loop until the end of the source string
    {
        *dest = *src; // copy the character from the source to the destination
        dest++; // move the pointer to the next character
        src++; // move the pointer to the next character
    }
    *dest = '\0'; // add a null character at the end of the destination string
    return temp; // return the original destination string
}

// A function to copy one string to another string
char *strcpy (char *dest, char *src)
{
    char *temp = dest; // a pointer to store the original destination string
    while (*src != '\0') // loop until the end of the source string
    {
        *dest = *src; // copy the character from the source to the destination
        dest++; // move the pointer to the next character
        src++; // move the pointer to the next character
    }
    *dest = '\0'; // add a null character at the end of the destination string
    return temp; // return the original destination string
}

```

Hello, I am Sydney, your AI assistant. I can help you with your topic. Here is the content in markdown format:

29. Define a structure data type TRAIN_INFO.
The type contain Train No.: integer type Train
name: string Departure Time: aggregate type
TIME Arrival Time: aggregate type TIME Start
station: string End station: string The structure
type Time contains two integer members: hour
and minute. Maintain a train timetable and im-
plement the following operations:

- A structure data type is a user-defined data type that can store a group of related values of different types.
- A structure data type TRAIN_INFO can be defined as follows:

```
// Define a structure type TIME
struct TIME
{
    int hour; // hour member
    int minute; // minute member
};

// Define a structure type TRAIN_INFO
struct TRAIN_INFO
{
    int train_no; // train number member
    char train_name[50]; // train name member
    struct TIME departure_time; // departure time member
    struct TIME arrival_time; // arrival time member
    char start_station[50]; // start station member
    char end_station[50]; // end station member
};
```

- To maintain a train timetable, we can declare an array of TRAIN_INFO structures and initialize it with some sample data:

```
// Declare an array of TRAIN_INFO structures
struct TRAIN_INFO timetable[5] = {
    {101, "Rajdhani Express", {10, 15}, {18, 30}, "New Delhi", "Mumbai"},
    {102, "Shatabdi Express", {8, 00}, {12, 00}, "Chennai", "Bangalore"},
    {103, "Durgam Express", {6, 30}, {14, 45}, "Kolkata", "Delhi"},
    {104, "Garib Rath", {9, 45}, {16, 15}, "Hyderabad", "Pune"},
    {105, "Jan Shatabdi", {7, 30}, {11, 30}, "Jaipur", "Agra"}
};
```

- To implement the following operations, we can use functions and loops to

manipulate the array of structures:

- Display the train timetable in a tabular format.
 - Search for a train by train number or train name and display its details.
 - Add a new train to the timetable.
 - Delete a train from the timetable by train number or train name.
 - Update the details of a train by train number or train name.
 - Sort the timetable by train number, train name, departure time, arrival time, start station or end station.
- Here is an example of how to display the train timetable in a tabular format:

```
// Define a function to display the train timetable
void display_timetable(struct TRAIN_INFO timetable[], int size)
{
    // Print the header row
    printf("Train No.\tTrain Name\tDeparture Time\tArrival Time\tStart Station\tEnd Station\n");

    // Loop through the array of structures and print each row
    for (int i = 0; i < size; i++)
    {
        printf("%d\t%s\t%d:%d\t%d:%d\t%s\t%s\n", timetable[i].train_no, timetable[i].train_name,
            timetable[i].departure_time / 100, timetable[i].departure_time % 100,
            timetable[i].arrival_time / 100, timetable[i].arrival_time % 100,
            timetable[i].start_station, timetable[i].end_station);
    }
}
```

- The output of this function would look like this:

Train No.	Train Name	Departure Time	Arrival Time	Start Station	End Station
101	Rajdhani Express	10:15	18:30	New Delhi	Mumbai
102	Shatabdi Express	8:00	12:00	Chennai	Bangalore
103	Duronto Express	6:30	14:45	Kolkata	Delhi
104	Garib Rath	9:45	16:15	Hyderabad	Pune
105	Jan Shatabdi	7:30	11:30	Jaipur	Agra

- I hope this helps you with your topic. If you have any questions or feedback, please let me know.

a. List all the trains (sorted according to train number) that depart from a particular section.

- To list all the trains that depart from a particular section, we need to use the **section** and **train** tables from the railway database.
- The **section** table contains information about the sections of the railway network, such as the section number, the starting station, the ending station, and the distance.
- The **train** table contains information about the trains that operate on the network, such as the train number, the train name, the source station, the

destination station, and the departure time.

- To find the trains that depart from a particular section, we need to join the two tables on the condition that the source station of the train matches the starting station of the section.
- To sort the trains according to the train number, we need to use the **order by** clause with the train number attribute in ascending order.
- The SQL query to list all the trains that depart from a particular section (say section 1) is:

```
select train.train_no, train.train_name, train.source, train.destination, train.departure
from section join train
on section.start = train.source
where section.section_no = 1
order by train.train_no;
```

- The output of the query will be a table with the following columns: train_no, train_name, source, destination, departure.
- The rows of the table will be the trains that depart from section 1, sorted by train number. For example:

train_no	train_name	source	destination	departure
101	Rajdhani	Delhi	Mumbai	08:00
102	Shatabdi	Delhi	Chandigarh	09:00
103	Duronto	Delhi	Kolkata	10:00
104	Garib Rath	Delhi	Patna	11:00
105	Jan Shatabdi	Delhi	Jaipur	12:00

b. List all the trains that depart from a particular station at a particular time.

- To list all the trains that depart from a particular station at a particular time, one can use the following steps:
 - Identify the station name and the time of departure that you are interested in. For example, you may want to find all the trains that depart from New Delhi station at 10:00 am.
 - Use a reliable online source or a printed timetable to search for the trains that match your criteria. For example, you may use the official website of Indian Railways or a third-party app like Trainman or RailYatri to look for the trains.
 - Filter the search results by selecting the station name and the time range that you want. For example, you may select New Delhi as the source station and 10:00 am to 10:30 am as the time range to narrow down the results.
 - Sort the results by the departure time or the train number to see the list of trains in a logical order. For example, you may sort the results

- by the departure time to see the trains that leave earliest to latest.
- Note down the train names, numbers, destinations, and other relevant details that you need. For example, you may note down the train name as Rajdhani Express, the train number as 12424, the destination as Dibrugarh, and the departure time as 10:10 am.
- Repeat the steps for any other station or time that you want to check. For example, you may want to compare the trains that depart from New Delhi and Old Delhi stations at 10:00 am.

c. List all the trains that depart from a particular station within the next one hour of a given time.

To list all the trains that depart from a particular station within the next one hour of a given time, one can follow these steps:

- Identify the station name and the given time.
- Access the online database or timetable of the trains that serve the station.
- Filter the trains by their departure time and status.
- Select the trains that depart within the next one hour of the given time and are not cancelled or delayed.
- Display the list of the selected trains with their departure time, destination, and platform number.

Alternatively, one can also use a mobile application or a website that provides real-time information on train departures from a particular station. For example, one can use the Trainline app or the National Rail website in the UK. These services allow the user to enter the station name and the given time, and then show the list of the trains that depart within the next one hour of the given time, along with their live status and other details.

d. List all the trains between a pair of start station and end station.

- To list all the trains between a pair of start station and end station, we need to use a database or a web service that contains the information about the train schedules, routes, and availability.
- One possible way to access such a database or a web service is to use an API (Application Programming Interface) that allows us to send queries and receive responses in a structured format, such as JSON (JavaScript Object Notation) or XML (Extensible Markup Language).
- For example, we can use the Indian Railways API (<https://indianrailapi.com/>) to list all the trains between two stations in India. The API requires an API key, which can be obtained by registering on the website.
- To use the API, we need to construct a URL (Uniform Resource Locator) that specifies the parameters of our query, such as the start station code, the end station code, the date of travel, and the API key. For example,

the URL for listing all the trains between New Delhi (NDLS) and Mumbai Central (BCT) on 15 March 2023 is:

<https://indianrailapi.com/api/v2/TrainBetweenStation/apikey//From/NDLS/To/BCT/Date/15-03-2023>

- The API will return a JSON response that contains an array of objects, each representing a train that matches our query. Each object will have properties such as TrainNo, TrainName, Source, Destination, DepartureTime, ArrivalTime, TravelTime, etc. For example, one possible object in the response is:

```
{ "TrainNo": "12952", "TrainName": "MUMBAI RAJDHANI", "Source":  
"NDLS", "Destination": "BCT", "DepartureTime": "16:25", "ArrivalTime":  
"08:15", "TravelTime": "15:50", "TrainType": "RAJDHANI", "Classes":  
[ { "ClassCode": "1A", "Availability": "AVAILABLE-0002" }, { "Class-  
Code": "2A", "Availability": "AVAILABLE-0010" }, { "ClassCode": "3A",  
"Availability": "AVAILABLE-0015" } ] }
```

- To list all the trains between the start station and the end station, we can iterate over the array of objects in the response and print or display the relevant properties of each object, such as TrainNo, TrainName, DepartureTime, ArrivalTime, etc. For example, the output of listing all the trains between New Delhi and Mumbai Central on 15 March 2023 could be:

TrainNo	TrainName	DepartureTime	ArrivalTime
12952	MUMBAI RAJDHANI	16:25	08:15
12954	AG KRANTI RJDHN	16:50	09:45
19024	FZR BCT JANTA	13:30	07:25
22210	NDLS BCT DURONTO	23:25	16:15

30. WAP to swap two elements using the concept of pointers.

- A pointer is a variable that stores the address of another variable in memory.
- To swap two elements using pointers, we need to pass the addresses of the elements to a function that will swap their values using a temporary variable.
- The function will use the dereference operator (*) to access the values pointed by the pointers and assign them to the temporary variable and vice versa.
- The function will not return anything, but the changes will be reflected in the original variables as they are passed by reference.
- Here is an example of a C program that swaps two integers using pointers:


```

#include <stdio.h>

// A function that swaps the values of two integers pointed by x and y
void swap(int *x, int *y)
{
    // Declare a temporary variable
    int temp;

    // Store the value pointed by x in temp
    temp = *x;

    // Assign the value pointed by y to the location pointed by x
    *x = *y;

    // Assign the value of temp to the location pointed by y
    *y = temp;
}

int main()
{
    // Declare and initialize two integers
    int a = 10, b = 20;

    // Print the original values of a and b
    printf("Before swapping: a = %d, b = %d\n", a, b);

    // Call the swap function and pass the addresses of a and b
    swap(&a, &b);

    // Print the swapped values of a and b
    printf("After swapping: a = %d, b = %d\n", a, b);

    return 0;
}

```

- The output of the program is:

Before swapping: a = 10, b = 20

After swapping: a = 20, b = 10

31. WAP to compare the contents of two files and determine whether they are same or not.

- To compare the contents of two files and determine whether they are same or not, we can use the following algorithm:
 - Open both files in read mode.
 - Initialize a flag variable to True.

- Loop through the lines of both files simultaneously using a zip function.
- Compare each line of the first file with the corresponding line of the second file using the == operator.
- If any line does not match, set the flag to False and break the loop.
- Close both files.
- Return the flag value as the result of the comparison.
- Here is an example of a Python program that implements this algorithm:

```
# Define a function to compare two files
def compare_files(file1, file2):
    # Open both files in read mode
    f1 = open(file1, "r")
    f2 = open(file2, "r")
    # Initialize a flag variable to True
    flag = True
    # Loop through the lines of both files simultaneously
    for line1, line2 in zip(f1, f2):
        # Compare each line of the first file with the corresponding line of the second file
        if line1 != line2:
            # If any line does not match, set the flag to False and break the loop
            flag = False
            break
    # Close both files
    f1.close()
    f2.close()
    # Return the flag value as the result of the comparison
    return flag

# Test the function with two sample files
file1 = "sample1.txt"
file2 = "sample2.txt"
result = compare_files(file1, file2)
if result:
    print("The files are same.")
else:
    print("The files are different.")
```

32.WAP to check whether a given word exists in a file or not. If yes then find the number of times it occurs.

- A possible solution to this problem is to use the `open()` function to read the file and the `count()` method to count the occurrences of the word in each line of the file.
- The `open()` function takes the name of the file as an argument and returns a file object that can be used to read or write the file.

- The `count()` method takes a substring as an argument and returns the number of times it appears in the string.
- The algorithm for the solution is as follows:
 - Declare a variable `word` to store the word to be searched and assign it a value.
 - Declare a variable `filename` to store the name of the file and assign it a value.
 - Declare a variable `count` to store the number of occurrences of the word and initialize it to zero.
 - Open the file using the `open()` function and assign the file object to a variable `file`.
 - Use a `for` loop to iterate over each line of the file.
 - Use the `count()` method to count the number of occurrences of the word in the current line and add it to the `count` variable.
 - Close the file using the `close()` method of the file object.
 - Check if the `count` variable is greater than zero.
 - If yes, print a message that the word exists in the file and the number of times it occurs.
 - If no, print a message that the word does not exist in the file.
- A possible implementation of the solution in Python is as follows:

```
# Declare the word to be searched
word = "hello"

# Declare the name of the file
filename = "sample.txt"

# Initialize the count to zero
count = 0

# Open the file
file = open(filename, "r")

# Loop over each line of the file
for line in file:
    # Count the occurrences of the word in the line
    count += line.count(word)

# Close the file
file.close()

# Check if the word exists in the file
if count > 0:
    # Print the message and the count
    print(f"The word '{word}' exists in the file '{filename}' and occurs {count} times.")
```

```

else:
    # Print the message
    print(f"The word '{word}' does not exist in the file '{filename}'.")

```

Note:

- A note is a brief piece of writing that records information or observations on a specific topic, usually for future reference or study.
- Notes can be taken from various sources, such as books, lectures, articles, videos, podcasts, etc.
- Notes can have different purposes, such as summarizing, paraphrasing, quoting, analyzing, synthesizing, evaluating, or applying the information.
- Notes can be organized in different ways, such as linear, outline, mind map, Cornell, chart, matrix, etc.
- Notes can be written in different formats, such as bullet points, sentences, keywords, symbols, abbreviations, etc.
- Notes can be improved by using different techniques, such as highlighting, color-coding, numbering, labeling, linking, etc.
- Notes can be reviewed and revised periodically to enhance memory and understanding of the information.

a) The Instructor may add/delete/modify/tune experiments, wherever he/she feels in a justified manner

- This statement implies that the instructor has the authority and responsibility to design and implement the experiments for the course, according to the learning objectives and outcomes.
- The instructor may add new experiments to introduce new concepts, skills, or applications that are relevant and useful for the course.
- The instructor may delete existing experiments to avoid redundancy, irrelevance, or obsolescence of the content or methods.
- The instructor may modify or tune the experiments to update, improve, or adapt them to the current context, requirements, or resources of the course.
- The instructor should always provide a clear and valid justification for any changes made to the experiments, and communicate them to the students and other stakeholders in a timely and transparent manner.
- The instructor should also ensure that the changes do not compromise the quality, rigor, or fairness of the assessment and evaluation of the experiments.

b) The subject teachers are suggested to use the concept of project based learning. The subject teacher may give certain use cases/case studies where student is able to apply multiple concepts in one single program

- Project based learning (PBL) is a teaching method that engages students in learning by solving real-world problems or challenges.
- PBL helps students develop 21st century skills such as critical thinking, creativity, collaboration, communication, and self-management.
- PBL also helps students connect their learning to their interests, passions, and future goals.
- PBL can be applied to any subject or discipline, but it is especially suitable for computer science, where students can use programming to create solutions for various scenarios or domains.
- Some examples of use cases/case studies for PBL in computer science are:
 - Creating a website or an app for a social cause, such as raising awareness, fundraising, or providing information or services to a specific community or group.
 - Developing a game or a simulation that teaches a concept, such as physics, math, history, or geography, or that entertains or challenges the player.
 - Designing and implementing an algorithm or a data structure that solves a problem, such as sorting, searching, encryption, compression, or optimization.
 - Analyzing and visualizing a data set that reveals a pattern, a trend, a correlation, or a prediction, such as weather, sports, health, or finance.
 - Building a hardware or a software system that interacts with the physical world, such as a robot, a drone, a sensor, or a smart device.
- To implement PBL in computer science, the subject teacher may follow these steps:
 - Identify the learning objectives and the standards that the project will address.
 - Choose a relevant and engaging topic or theme that sparks students' curiosity and interest.
 - Define the problem or the challenge that the project will aim to solve or address.
 - Provide the students with the necessary resources, tools, and guidance to conduct research, brainstorm ideas, plan, design, and develop their solutions.
 - Facilitate the students' collaboration, feedback, and reflection throughout the project process.

- Assess the students’ learning outcomes and their products based on a rubric or a checklist that measures their knowledge, skills, and attitudes.
- Showcase the students’ work and celebrate their achievements.

c) It is also suggested that open source tools should be preferred to conduct the lab. Some open source online compiler to conduct the C lab are as follows:

- **Repl.it:** This is a web-based IDE that supports multiple programming languages, including C. It allows users to create, run, and share code snippets online. It also has features such as syntax highlighting, code formatting, debugging, and collaboration. Repl.it is free to use for basic features, but requires a subscription for advanced features such as private repls, version control, and cloud storage. Repl.it can be accessed at <https://repl.it/languages/c>.
- **OnlineGDB:** This is another web-based IDE that supports C and other languages. It has features such as code editor, compiler, debugger, and terminal. It also allows users to save and share their code online. OnlineGDB is free to use and does not require registration. OnlineGDB can be accessed at https://www.onlinegdb.com/online_c_compiler.
- **JDoodle:** This is a simple online compiler that supports C and other languages. It allows users to write, compile, and execute code online. It also has features such as code formatting, input/output, and file upload. JDoodle is free to use and does not require registration. JDoodle can be accessed at <https://www.jdoodle.com/c-online-compiler>.

<https://www.jdoodle.com/c-online-compiler/>

- <https://www.jdoodle.com/c-online-compiler/> is a website that allows users to write, compile, run and debug C programs online.
- It is one of the online compilers and IDEs supported by JDoodle.com, a platform that serves the programming community with online tools since 2013 .
- It has features such as syntax highlighting, auto-completion, code formatting, interactive mode, file upload, stdin/stdout support, code sharing, collaboration, and more.
- It also supports multiple versions of C compilers, such as GCC 4.8.1, GCC 5.4.0, GCC 6.3.0, GCC 7.3.0, GCC 8.2.0, GCC 9.2.0, GCC 10.2.0, GCC 11.1.0, and Clang 3.8.0.
- It is useful for learning C programming, testing code snippets, debugging errors, and developing small projects.
- It also provides APIs and plugins for integrating the online compiler/IDE capability into other websites or applications .

- It is free to use for personal and educational purposes, but requires a subscription for commercial or professional use.

Online C Compiler - tutorialspoint.com

Online C Compiler is a web-based tool that allows you to write, compile, run and debug C programs online. It is provided by Tutorialspoint, a website that offers free tutorials on various programming languages and technologies.

Some features of Online C Compiler are:

- It supports C11 standard and has code highlighting, auto-completion and error detection features.
- It allows you to create, save, download and share your C projects online. You can also upload existing C files or projects to the compiler.
- It has a built-in terminal and a debugger that lets you execute and debug your C programs step by step.
- It has a variety of themes and settings that you can customize according to your preference.
- It has a help section that provides useful information and examples on C programming.

Some advantages of using Online C Compiler are:

- It is easy to use and does not require any installation or configuration on your system.
- It is accessible from any device and browser that has an internet connection.
- It is useful for learning and practicing C programming without the hassle of setting up a local environment.
- It is helpful for testing and debugging your C programs quickly and efficiently.

Some limitations of Online C Compiler are:

- It may not support some advanced features or libraries of C programming that are available in other compilers or IDEs.
- It may not be suitable for developing large or complex C projects that require more resources or security.
- It may not be compatible with some browsers or devices that have low performance or compatibility issues.

Online C Compiler

- An online C compiler is a web-based tool that allows you to write, compile, and run C programs online.

- It is useful for learning C programming, testing code snippets, or prototyping applications without installing any software on your local machine.
- There are many online C compilers available on the internet, each with different features and capabilities.
- Some of the common features of online C compilers are:
 - Syntax highlighting: The editor highlights the keywords, variables, operators, and other elements of the C language with different colors and fonts to make the code more readable and understandable.
 - Code formatting: The editor automatically formats the code according to the standard coding conventions, such as indentation, spacing, and braces.
 - Code completion: The editor suggests possible completions for the code as you type, based on the context and the available libraries.
 - Code debugging: The compiler detects and reports any errors or warnings in the code, such as syntax errors, type mismatches, or undefined variables. Some compilers also provide a debugger that allows you to step through the code, inspect the values of variables, and set breakpoints.
 - Code execution: The compiler runs the code and displays the output on the screen, either in a separate window or in the same editor. Some compilers also allow you to provide input to the program, either through the keyboard or through a file.
 - Code sharing: The compiler generates a unique URL for the code that you can share with others, either for collaboration or for demonstration purposes.
- Some of the popular online C compilers are:
 - OnlineGDB: It supports GCC compiler for C and C++, and provides a full-featured online debugger with breakpoints, watch variables, and console output.
 - GeeksforGeeks: It supports C, C++, C++14, C#, Java, Perl, PHP, Python, Python 3, Scala, and Node JS languages, and provides a simple and fast online editor with code execution and sharing features.
 - Programiz: It supports C and C++ languages, and provides a user-friendly online editor with code formatting, completion, debugging, execution, and sharing features. It also provides tutorials and examples for learning C and C++ programming.

HackerRank

HackerRank is a website that provides a platform for programmers to practice their skills and prepare for coding interviews. It also helps companies to assess the technical abilities of candidates and hire the best talent. Some of the features

of HackerRank are:

- It offers a variety of **skills**, **tracks** and **tutorials** for learners to explore different topics such as algorithms, data structures, machine learning, artificial intelligence, functional programming and more .
- It allows users to take **skills certification tests** and earn badges that showcase their proficiency in various programming languages and domains.
- It hosts **code challenges** and **hackathons** where users can compete with other programmers from all over the world and win prizes .
- It provides a **developer dashboard** where users can track their progress, view their performance statistics, access their submissions and certificates, and customize their profile.
- It enables companies to create **custom assessments** and **interviews** that test the specific skills and knowledge required for their roles .
- It integrates with **popular tools** such as GitHub, LinkedIn, Slack, Workday, Greenhouse and more to streamline the hiring process and enhance the candidate experience .

HackerRank is a critical piece of our candidate's experience, and hitting the right tone is just as important as identifying good candidates. HackerRank has been a huge help in making it easy to assess the skills of all the candidates we interview. Without it, the process of scaling our engineering team would have been very cumbersome.

Mapping with Virtual Lab

- Mapping with virtual lab is a process of creating isolated networks that resemble the production networks in a virtual environment.
- Mapping with virtual lab allows testing, troubleshooting, and training scenarios without affecting the production environment or changing the IP addresses of the virtual machines (VMs).
- Mapping with virtual lab requires a proxy appliance that acts as a gateway between the production network and the isolated network.
- Mapping with virtual lab can be done in different modes, such as basic, advanced, or masquerade, depending on the level of isolation and accessibility required.
- Mapping with virtual lab involves the following steps:
 - Launching a new virtual lab wizard and specifying the name, description, host, and datastore for the virtual lab.
 - Setting up the proxy appliance and selecting the networking mode.
 - Creating isolated networks and mapping them to the corresponding production networks.
 - Specifying the network settings and static IP mapping rules for the isolated networks.
 - Applying the parameters and connecting to the virtual lab.
- Mapping with virtual lab can be used for various purposes, such as:

- Testing backup and restore operations without affecting the production data.
- Troubleshooting network issues or configuration changes in a safe environment.
- Training students or employees on how to use specific applications or systems.

Name of the Lab: Physics Lab

Name of the Experiment: Determination of the focal length of a convex lens by the u-v method.

- The aim of this experiment is to determine the focal length of a convex lens by measuring the object distance (u) and the image distance (v) for different positions of the object.
- The principle of this experiment is based on the thin lens formula: $1/f = 1/u + 1/v$, where f is the focal length of the lens, u is the object distance and v is the image distance.
- The apparatus required for this experiment are: a convex lens, an optical bench, a light source, a screen, a meter scale and a needle.
- The procedure of this experiment is as follows:
 - Fix the light source at one end of the optical bench and the screen at the other end.
 - Place the convex lens between the light source and the screen and adjust its position until a sharp image of the light source is formed on the screen.
 - Measure the distance between the lens and the light source (u) and the distance between the lens and the screen (v) using the meter scale. Record these values in a table.
 - Repeat the above steps by changing the position of the lens and obtaining different values of u and v . At least six sets of readings should be taken.
 - Calculate the reciprocal of u ($1/u$) and the reciprocal of v ($1/v$) for each set of readings and tabulate them.
 - Plot a graph of $1/v$ versus $1/u$ on a graph paper. The slope of the graph gives the value of $1/f$.
 - Find the average value of $1/f$ from the graph and calculate the focal length of the lens as $f = 1/(1/f)$.
- The precautions to be taken while performing this experiment are:
 - The lens should be free from dust and scratches.
 - The image formed on the screen should be clear and distinct.
 - The meter scale should be placed parallel to the optical bench and the readings should be taken with minimum parallax error.

- The graph should be plotted with proper scales and units.

Problem Solving Lab

- The problem solving lab is a course that aims to develop the skills and strategies for solving complex and open-ended problems in various domains.
- The course covers the following topics:
 - Problem definition and analysis: how to identify, understand, and decompose a problem into subproblems, and how to use different types of information and data to support the problem solving process.
 - Problem solving methods and tools: how to apply various methods and tools for generating, evaluating, and selecting solutions, such as brainstorming, mind mapping, decision matrix, SWOT analysis, etc.
 - Problem solving communication and presentation: how to communicate and present the problem and the solution effectively to different audiences, using appropriate formats and media, such as reports, slides, posters, etc.
- The course consists of lectures, tutorials, and lab sessions, where students will work on individual and group projects that involve solving real-world problems from different disciplines and contexts.
- The course objectives are to:
 - Develop the ability to define and analyze complex and open-ended problems systematically and creatively.
 - Enhance the skills and confidence to apply various problem solving methods and tools to generate, evaluate, and select solutions.
 - Improve the communication and presentation skills to convey the problem and the solution clearly and persuasively.
 - Foster the collaboration and teamwork skills to work effectively with others in problem solving situations.

Numerical Representation

Numerical representation is the internal representation of numeric values in digital device hardware and software, such as in programmable computers and calculators. Numerical values are stored as groupings of bits, such as bytes and words. The encoding between numerical values and bit patterns is chosen for convenience of the operation of the computer; the encoding used by the computer's instruction set generally requires conversion for external use, such as for printing and display.

Some of the main topics related to numerical representation are:

- **Number systems:** Different ways of representing numbers using symbols and rules. The most common number systems are decimal (base 10), binary (base 2), octal (base 8) and hexadecimal (base 16). Each number

system has its advantages and disadvantages for different applications and purposes.

- **Integer representation:** How to store and manipulate whole numbers using bits. There are different methods of representing integers, such as unsigned, signed magnitude, one's complement and two's complement. Each method has its own range, arithmetic operations and overflow detection.
- **Floating-point representation:** How to store and manipulate real numbers (numbers with fractional parts) using bits. There are different standards of representing floating-point numbers, such as IEEE 754 and IBM 370. Each standard has its own format, precision, range, rounding modes and special values .
- **Computer algebra:** How to perform symbolic and exact computations with numbers and other mathematical objects, such as polynomials, matrices, functions and equations. Computer algebra systems use different data structures and algorithms to represent and manipulate algebraic expressions, such as lists, trees, graphs and hash tables. Computer algebra can avoid the approximation and loss of significant digits that occur in numerical computation .

Beauty of Numbers

- Beauty of numbers is the aesthetic pleasure derived from the abstractness, purity, simplicity, depth or orderliness of mathematics.
- Numbers can help us discover beautiful patterns in nature, such as the Fibonacci sequence, which is a series of numbers where each number is the sum of the previous two.
- The Fibonacci sequence can be found in many natural phenomena, such as the arrangement of petals in flowers, the spirals of sunflower seeds, the scales of pineapples, the shells of snails, and the branching of trees.
- Another example of beauty in numbers is the golden ratio, which is a special number that is approximately equal to 1.618. The golden ratio is also related to the Fibonacci sequence, as the ratio of consecutive Fibonacci numbers tends to the golden ratio as the sequence grows.
- The golden ratio is considered to be aesthetically pleasing and has been used in art, architecture, music, and design for centuries. Some examples of the golden ratio in art are the Mona Lisa, the Parthenon, and the Great Pyramid of Giza.
- A third example of beauty in numbers is the concept of symmetry, which is the property of being invariant under some transformation, such as reflection, rotation, or translation. Symmetry can be found in many mathematical objects, such as shapes, equations, functions, and patterns.
- Symmetry can also be found in nature, such as the bilateral symmetry of animals, the radial symmetry of flowers, the rotational symmetry of snowflakes, and the translational symmetry of crystals.
- A fourth example of beauty in numbers is the idea of fractals, which

are self-similar patterns that repeat at different scales. Fractals can be generated by simple mathematical rules, such as the Mandelbrot set, which is the set of complex numbers that do not escape to infinity under repeated iteration of a function.

- Fractals can also be found in nature, such as the coastlines of continents, the shapes of mountains, the branching of veins, the structure of lungs, and the patterns of lightning.
- These are just some of the examples of the beauty of numbers in nature and mathematics. There are many more to explore and appreciate, as numbers can reveal the hidden order, harmony, and elegance of the universe.

More on Numbers

- Numbers are symbols that represent quantities or values.
- There are different types of numbers, such as natural numbers, whole numbers, integers, rational numbers, irrational numbers, real numbers, and complex numbers.
- Natural numbers are the counting numbers, such as 1, 2, 3, 4, and so on. They are also called positive integers.
- Whole numbers are the natural numbers and zero, such as 0, 1, 2, 3, 4, and so on.
- Integers are the whole numbers and their negatives, such as -4, -3, -2, -1, 0, 1, 2, 3, 4, and so on.
- Rational numbers are the numbers that can be written as a fraction of two integers, such as $\frac{1}{2}$, $\frac{3}{4}$, $-\frac{5}{6}$, 0.25, 1.5, and so on. They can also be written as terminating or repeating decimals.
- Irrational numbers are the numbers that cannot be written as a fraction of two integers, such as $\sqrt{2}$, π , e, and so on. They can only be written as non-terminating and non-repeating decimals.
- Real numbers are the numbers that can be represented on a number line, such as rational and irrational numbers. They are also called the set of all decimals.
- Complex numbers are the numbers that have a real part and an imaginary part, such as $2 + 3i$, $-4 - 5i$, $0 + i$, and so on. The imaginary part is a multiple of i , where i is the square root of -1. Complex numbers can be represented on a complex plane, where the horizontal axis is the real part and the vertical axis is the imaginary part.

Factorials

- A factorial is a mathematical operation that calculates the product of all positive integers from 1 to a given number n .
- The factorial of n is denoted by $n!$ and is defined as $n! = 1 \times 2 \times 3 \times \dots \times n$.
- For example, $5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$.
- The factorial of 0 is defined as $0! = 1$ by convention.

- Factorials are used to count the number of ways to arrange or order a set of objects, such as permutations and combinations.
- Factorials also appear in various formulas and identities in mathematics, such as the binomial theorem, Taylor series, and Stirling's approximation.
- Factorials grow very fast as n increases. For example, $10! = 3,628,800$ and $20! = 2,432,902,008,176,640,000$.
- The largest factorial that can be stored in a 64-bit integer is $20!$, since $21!$ exceeds the maximum value of $2^{63} - 1$.
- To calculate larger factorials, one can use arbitrary-precision arithmetic or special algorithms, such as the prime-swing algorithm or the gamma function.

String Operations

- A string is a sequence of characters enclosed in quotation marks, such as "Hello" or "Python".
- Strings can be concatenated (joined) using the $+$ operator, such as "Hello" + "World" = "HelloWorld".
- Strings can be repeated using the $*$ operator, such as "Hello" * 3 = "HelloHelloHello".
- Strings can be accessed by indexing, which returns a single character, such as "Hello"[0] = "H".
- Strings can be sliced, which returns a substring, such as "Hello"[1:3] = "el".
- Strings can be compared using the $==$ operator, which returns True if the strings are equal, and False otherwise, such as "Hello" == "Hello" = True, and "Hello" == "World" = False.
- Strings can be converted to other data types using built-in functions, such as $\text{int}("123") = 123$, and $\text{float}("3.14") = 3.14$.
- Strings have many methods that perform various operations on them, such as:
 - `upper()`, which returns a copy of the string in uppercase, such as "Hello".upper() = "HELLO".
 - `lower()`, which returns a copy of the string in lowercase, such as "Hello".lower() = "hello".
 - `replace(old, new)`, which returns a copy of the string with all occurrences of old replaced by new, such as "Hello".replace("l", "x") = "Hexxo".
 - `find(sub)`, which returns the index of the first occurrence of sub in the string, or -1 if not found, such as "Hello".find("l") = 2, and "Hello".find("z") = -1.
 - `split(sep)`, which returns a list of substrings separated by sep, such as "Hello,World".split(",") = ["Hello", "World"].
 - `join(iterable)`, which returns a string that is the concatenation of the elements in iterable, separated by the string itself, such as "-".join(["Hello", "World"]) = "Hello-World".

- `strip(chars)`, which returns a copy of the string with leading and trailing characters removed, such as `" Hello ".strip() == "Hello"`, and `"Hello".strip("H") == "ello"`.
- `format(*args, **kwargs)`, which returns a formatted version of the string, replacing placeholders with values from `args` and `kwargs`, such as `"Hello, {name}".format(name="World") == "Hello, World"`.

Recursion

Recursion is a technique of defining a problem in terms of itself. It is a way of solving complex problems by breaking them down into smaller and simpler subproblems that have the same structure as the original problem.

Recursion involves two main components:

- A **base case** or **terminating condition** that defines the simplest version of the problem and provides a direct solution.
- A **recursive step** or **recursive call** that reduces the problem to a smaller and simpler subproblem, and then calls itself with the new subproblem as the input.

Recursion can be used to implement algorithms that are naturally recursive, such as:

- Searching and sorting algorithms (e.g., binary search, merge sort, quick sort)
- Tree and graph traversal algorithms (e.g., depth-first search, breadth-first search, backtracking)
- Mathematical computations (e.g., factorial, Fibonacci, exponentiation, Euclidean algorithm)
- Dynamic programming and memoization (e.g., longest common subsequence, knapsack problem, coin change problem)

Recursion has some advantages and disadvantages over iterative solutions:

- Advantages:
 - Recursion can make the code more concise, elegant, and readable.
 - Recursion can express the logic of the problem more naturally and intuitively.
 - Recursion can avoid the use of explicit loops and variables.
- Disadvantages:
 - Recursion can cause more memory and time overhead, as each recursive call creates a new stack frame and may perform redundant computations.
 - Recursion can cause stack overflow errors if the base case is not reached or the recursive depth is too large.
 - Recursion can be harder to debug and trace.

Advanced Arithmetic

Advanced arithmetic is the branch of mathematics that deals with operations on numbers beyond the basic four: addition, subtraction, multiplication and division. Some of the topics covered in advanced arithmetic are:

- Exponents and logarithms: These are ways of expressing repeated multiplication or division using a base and an exponent or a logarithm. For example, $2^3 = 2 \times 2 \times 2 = 8$ and $\log_2(8) = 3$.
- Roots and radicals: These are ways of expressing fractional exponents or inverse operations of exponents. For example, $8^{(1/3)} = 2$ and $2^{(3/2)} = \sqrt{2^3} = \sqrt{8}$.
- Fractions and decimals: These are ways of expressing parts of a whole or ratios using numerators and denominators or decimal points. For example, $3/4 = 0.75$ and $0.6 = 6/10$.
- Percentages and proportions: These are ways of expressing fractions or decimals as parts of 100 or as ratios of two quantities. For example, $75\% = 75/100 = 0.75$ and $3/4 = 9/12$.
- Ratios and rates: These are ways of expressing comparisons or relationships between two quantities using fractions or division. For example, $3:4 = 3/4$ and $60 \text{ km/h} = 60/1 \text{ km per hour}$.
- Unit conversions: These are ways of expressing the same quantity in different units using multiplication or division by conversion factors. For example, $1 \text{ m} = 100 \text{ cm}$ and $1 \text{ kg} = 1000 \text{ g}$.
- Scientific notation: This is a way of expressing very large or very small numbers using powers of 10 and a coefficient between 1 and 10. For example, 6.02×10^{23} and 3.14×10^{-7} .
- Order of operations: This is a set of rules that determines the order in which arithmetic operations are performed in an expression. The common acronym for the order of operations is PEMDAS, which stands for parentheses, exponents, multiplication and division, addition and subtraction. For example, $2 + 3 \times 4 = 2 + 12 = 14$ and $(2 + 3) \times 4 = 5 \times 4 = 20$.

Searching and Sorting

Searching and sorting are two fundamental operations in computer science. They are used to manipulate and organize data in various ways. Searching is the process of finding a specific element or a subset of elements in a collection of data, while sorting is the process of arranging the elements of a collection in a specific order.

Some of the common applications of searching and sorting are:

- Finding a word in a dictionary
- Looking up a phone number in a contact list
- Sorting a list of names alphabetically
- Finding the best route to a destination
- Sorting a collection of photos by date or location

There are different algorithms and techniques for searching and sorting data, depending on the type, size, and structure of the data, as well as the desired efficiency and accuracy of the operation. Some of the factors that affect the performance of searching and sorting algorithms are:

- Time complexity: the amount of time required to complete the operation, usually measured by the number of comparisons or swaps performed
- Space complexity: the amount of extra memory required to perform the operation, usually measured by the number of auxiliary variables or arrays used
- Stability: the property of preserving the relative order of equal elements after sorting
- Adaptability: the ability to perform better on partially sorted or nearly sorted data
- In-place: the property of not using any extra memory to perform the operation

Some of the common searching and sorting algorithms are:

- Linear search: a simple algorithm that scans the data sequentially from left to right until the target element is found or the end of the data is reached. It has a time complexity of $O(n)$ and a space complexity of $O(1)$. It is not stable, adaptable, or in-place.
- Binary search: an efficient algorithm that works on sorted data by repeatedly dividing the data into two halves and comparing the target element with the middle element of each half. It has a time complexity of $O(\log n)$ and a space complexity of $O(1)$. It is not stable, adaptable, or in-place.
- Selection sort: a simple algorithm that sorts the data by repeatedly finding the smallest or largest element in the unsorted part of the data and swapping it with the first or last element of the unsorted part. It has a time complexity of $O(n^2)$ and a space complexity of $O(1)$. It is not stable, adaptable, or in-place.
- Insertion sort: an efficient algorithm that sorts the data by repeatedly inserting the next element in the unsorted part of the data into its correct position in the sorted part of the data. It has a time complexity of $O(n^2)$ in the worst case and $O(n)$ in the best case, and a space complexity of $O(1)$. It is stable, adaptable, and in-place.
- Bubble sort: a simple algorithm that sorts the data by repeatedly swapping adjacent elements that are out of order until no swaps are needed. It has a time complexity of $O(n^2)$ in the worst case and $O(n)$ in the best case, and a space complexity of $O(1)$. It is stable, adaptable, and in-place.
- Merge sort: a recursive algorithm that sorts the data by dividing it into two halves, sorting each half recursively, and then merging the two sorted halves. It has a time complexity of $O(n \log n)$ and a space complexity of $O(n)$. It is stable, not adaptable, and not in-place.
- Quick sort: a recursive algorithm that sorts the data by choosing a pivot element, partitioning the data into two parts such that all elements less

than the pivot are in the left part and all elements greater than or equal to the pivot are in the right part, and then sorting each part recursively. It has a time complexity of $O(n^2)$ in the worst case and $O(n \log n)$ in the average case, and a space complexity of $O(\log n)$. It is not stable, not adaptable, and in-place.

- Heap sort: an algorithm that sorts the data by using a data structure called a heap, which is a complete binary tree that satisfies the heap property, meaning that each node is greater than or equal to its children. It has a time complexity of $O(n \log n)$ and a space complexity of $O(1)$. It is not stable, not adaptable, and in-place.

These are some of the basic concepts and algorithms of searching and sorting. There are many more variations and optimizations that can be applied to different scenarios and data types. Searching and sorting are essential skills for any computer scientist or programmer to master.

Permutation

- A permutation is an arrangement of objects in a specific order.
- The order of the objects matters in a permutation.
- For example, the permutations of the letters A, B, and C are ABC, ACB, BAC, BCA, CAB, and CBA. Changing the order of the letters produces different permutations.
- The number of permutations of n distinct objects is n factorial, denoted by $n!$.
- $n! = n * (n-1) * (n-2) * \dots * 3 * 2 * 1$
- For example, the number of permutations of 3 distinct objects is $3! = 3 * 2 * 1 = 6$.
- If some of the objects are repeated, the number of permutations is reduced by dividing by the factorial of the number of repetitions.
- For example, the number of permutations of the letters A, A, and B is $3! / 2! = 3$, because there are 2 repetitions of A.
- The permutations of A, A, and B are AAB, ABA, and BAA.
- A permutation of r objects chosen from n distinct objects is called a permutation of n objects taken r at a time, denoted by $P(n, r)$.
- $P(n, r) = n! / (n-r)!$
- For example, the number of permutations of 2 letters chosen from 4 distinct letters is $P(4, 2) = 4! / (4-2)! = 12$.
- The permutations of 2 letters chosen from A, B, C, and D are AB, AC, AD, BA, BC, BD, CA, CB, CD, DA, DB, and DC.

Sequences

- A sequence is a list of numbers or objects that follow a certain pattern or rule.
- A sequence can be finite or infinite, depending on whether it has a fixed

number of terms or not.

- A term is an element or a member of a sequence. The position of a term in a sequence is called its index or subscript.
- A sequence can be defined by a formula that gives the n th term as a function of n , or by a recursive rule that gives the next term based on the previous terms, or by listing the terms explicitly.
- Examples of sequences are:
 - The arithmetic sequence: $a, a+d, a+2d, a+3d, \dots$, where a is the first term and d is the common difference.
 - The geometric sequence: $a, ar, ar^2, ar^3, \dots$, where a is the first term and r is the common ratio.
 - The Fibonacci sequence: $1, 1, 2, 3, 5, 8, 13, \dots$, where each term is the sum of the previous two terms.
 - The harmonic sequence: $1, 1/2, 1/3, 1/4, \dots$, where each term is the reciprocal of its index.
 - The prime numbers: $2, 3, 5, 7, 11, 13, 17, \dots$, where each term is a natural number that has exactly two factors, 1 and itself.
- A sequence can be represented by a notation that uses curly braces and commas, such as $\{a, a+d, a+2d, \dots\}$, or by using a subscript notation, such as a_n , where n is the index of the term.
- A sequence can be graphed by plotting its terms on a coordinate plane, where the x -axis is the index and the y -axis is the value of the term. The graph of a sequence is a set of discrete points, not a continuous curve.

Course Outcomes:

- Course outcomes are statements that describe what students are expected to know and be able to do by the end of a course.
- Course outcomes are aligned with the course objectives, which are derived from the program outcomes and the institutional mission and vision.
- Course outcomes are measurable, observable, and achievable within the duration and scope of the course.
- Course outcomes are written in terms of student learning, using action verbs that indicate the level of cognitive skills required.
- Course outcomes are used to guide the design of the course content, activities, assessments, and feedback.
- Course outcomes are communicated to the students at the beginning of the course and throughout the course as a way of clarifying the expectations and the purpose of the course.
- Course outcomes are evaluated at the end of the course to determine the extent to which the students have achieved the desired learning outcomes and to identify areas for improvement.

Course Outcome Bloom's Taxonomy

- Course outcome is a brief statement that describes what students will be expected to learn by the end of the course.
- Bloom's taxonomy is a model of cognitive skills used to classify educational learning objectives and is a helpful tool for the development of learning outcomes .
- Bloom's taxonomy consists of six levels of thinking, from lower-order to higher-order: remember, understand, apply, analyze, evaluate, and create .
- The action verbs used in Bloom's taxonomy are measurable and discrete, and they indicate the level of rigor intended for the course .
- When writing learning outcomes, it is important to use Bloom's taxonomy to make sure that the verbs match the level of the course and the learning objectives .
- The process outlined in Bloom's taxonomy provides a scaffolding around which instructors can design their course, and it also explains the process of learning: before students can understand a concept, they must remember it; to apply a concept, they must first understand it; and so on.
- Aim for learning outcomes that include skills that span across the pyramid of Bloom's taxonomy. Although it is easy to focus on the foundations of "remember" and "understand", try to include at least one outcome that strives for "evaluation" or "creation", which are the highest levels of thinking.

Level

- A level is a measure of the amount or degree of something, such as height, quantity, quality, intensity, etc.
- A level can also refer to a position or rank in a hierarchy, such as a level of authority, education, skill, etc.
- A level can also refer to a stage or phase in a process, such as a level of difficulty, progress, development, etc.
- A level can also refer to a flat or horizontal surface or plane, such as a level of a building, a level of a game, a level of a field, etc.
- A level can also refer to a device or instrument that indicates whether a surface is horizontal or vertical, such as a spirit level, a water level, a laser level, etc.

At the end of course, the student will be able to:

- Define the basic concepts and principles of artificial intelligence, such as agents, environments, rationality, search, knowledge representation, reasoning, planning, learning, and natural language processing.
- Apply various search algorithms, such as uninformed search, informed search, local search, and adversarial search, to solve different kinds of

problems, such as pathfinding, puzzle solving, game playing, and constraint satisfaction.

- Represent and manipulate knowledge using different formalisms, such as propositional logic, first-order logic, semantic networks, frames, and ontologies, and use inference methods, such as resolution, forward chaining, and backward chaining, to draw conclusions from knowledge bases.
- Design and implement planning systems that can generate sequences of actions to achieve goals, using different planning techniques, such as state-space planning, hierarchical planning, partial-order planning, and plan execution and monitoring.
- Understand and apply the basic concepts and techniques of machine learning, such as supervised learning, unsupervised learning, reinforcement learning, decision trees, neural networks, clustering, and dimensionality reduction, to various domains and tasks, such as classification, regression, clustering, and dimensionality reduction.
- Analyze and process natural language texts and speech using different methods and tools, such as regular expressions, finite-state automata, context-free grammars, parsing, semantic analysis, and natural language generation.

CO 1 Able to implement the algorithms and draw flowcharts for solving Mathematical and Engineering problems.

- An algorithm is a step-by-step procedure to solve a problem or perform a task.
- A flowchart is a graphical representation of an algorithm using symbols and arrows to show the sequence of steps and the logic of the solution.
- Algorithms and flowcharts are useful tools for designing, analyzing, and implementing solutions for mathematical and engineering problems.
- Some examples of mathematical and engineering problems that can be solved using algorithms and flowcharts are:
 - Finding the roots of a quadratic equation.
 - Sorting an array of numbers in ascending or descending order.
 - Computing the factorial of a positive integer.
 - Finding the greatest common divisor of two numbers.
 - Converting a decimal number to binary or vice versa.
 - Solving a system of linear equations using matrix operations.
 - Finding the shortest path between two nodes in a graph.
 - Encrypting or decrypting a message using a cipher.
- To implement an algorithm and draw a flowchart for solving a problem, one should follow these steps:

- Understand the problem and its requirements.
- Identify the input and output data and their formats.
- Break down the problem into smaller and simpler subproblems.
- Design an algorithm for each subproblem using pseudocode or natural language.
- Test and debug the algorithm using sample input and output data.
- Draw a flowchart for the algorithm using standard symbols and conventions.
- Implement the algorithm using a programming language of choice.
- Run and evaluate the program using different input and output data.

K3, K4

- K3 and K4 are two types of **knowledge representation languages** that are used to encode knowledge in a declarative and logical way.
- K3 is a **first-order logic** language that allows expressing facts, rules, and queries using predicates, variables, constants, and logical connectives.
- K4 is a **description logic** language that allows expressing concepts, roles, and individuals using classes, properties, and instances, and logical operators.
- K3 and K4 have different syntax and semantics, but they can be **inter-translated** using a set of rules that preserve the meaning and structure of the knowledge.
- K3 and K4 are both **decidable** languages, meaning that there is an algorithm that can determine whether a given statement is true or false in a finite amount of time.
- K3 and K4 are both **expressive** languages, meaning that they can capture a wide range of knowledge domains and reasoning tasks.
- K3 and K4 are both **tractable** languages, meaning that the computational complexity of answering queries is polynomial in the size of the knowledge base.

CO 2 Demonstrate an understanding of computer programming language concepts. K3, K2

- Computer programming language is a set of rules and symbols that instruct a computer to perform specific tasks.
- Computer programming language concepts are the fundamental ideas and principles that underlie the design and implementation of programming languages.
- Some of the common computer programming language concepts are:
 - Syntax: The rules that define how to write valid statements and expressions in a programming language.

- Semantics: The meaning and behavior of the statements and expressions in a programming language.
- Data types: The categories of values that can be stored and manipulated by a programming language, such as numbers, strings, booleans, arrays, etc.
- Variables: The names that refer to data values in a programming language, such as `x`, `y`, `name`, etc.
- Operators: The symbols that perform arithmetic, logical, or other operations on data values in a programming language, such as `+`, `-`, `*`, `/`, `&&`, `||`, etc.
- Control structures: The statements that control the flow of execution in a programming language, such as `if`, `else`, `for`, `while`, `switch`, etc.
- Functions: The blocks of code that perform a specific task and can be reused in a programming language, such as `print`, `sqrt`, `max`, etc.
- Parameters: The variables that are passed to a function to provide input or output values in a programming language, such as `x`, `y`, `z`, etc.
- Scope: The region of code where a variable is visible and accessible in a programming language, such as `global`, `local`, `block`, etc.
- Recursion: The technique of defining a function in terms of itself in a programming language, such as `factorial`, `fibonacci`, etc.
- Object-oriented programming: The paradigm of organizing data and behavior into classes and objects in a programming language, such as `Java`, `C++`, `Python`, etc.
- Class: The blueprint that defines the attributes and methods of a type of object in a programming language, such as `Person`, `Animal`, `Car`, etc.
- Object: The instance of a class that has its own state and behavior in a programming language, such as `p1`, `a1`, `c1`, etc.
- Inheritance: The mechanism of deriving a new class from an existing class in a programming language, such as `Dog` inherits from `Animal`, etc.
- Polymorphism: The ability of an object to behave differently depending on its type or context in a programming language, such as a `Dog` object can bark or wag its tail, etc.
- Abstraction: The process of hiding the details and complexity of a system and providing a simple and relevant interface in a programming language, such as using a function or a class, etc.
- Encapsulation: The principle of bundling data and behavior together and restricting access to them in a programming language, such as using `private` and `public` modifiers, etc.
- Functional programming: The paradigm of treating computation as the evaluation of mathematical functions and avoiding side effects in a programming language, such as `Haskell`, `Lisp`, etc.
- Higher-order function: The function that can take another function as an argument or return a function as a result in a programming

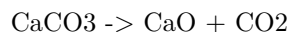
- language, such as map, filter, reduce, etc.
- Lambda expression: The anonymous function that can be defined and used inline in a programming language, such as $(x) \rightarrow x * x$, etc.
 - Closure: The function that can access and modify the variables in its enclosing scope in a programming language, such as `function(x) { return function(y) { return x + y; }; }, etc.`
 - Logic programming: The paradigm of expressing problems and solutions in terms of facts and rules in a programming language, such as Prolog, etc.
 - Fact: The statement that declares a relation between terms in a logic programming language, such as `parent(john, mary)`, etc.
 - Rule: The statement that defines a relation between terms in terms of other relations in a logic programming language, such as `grandparent(X, Y) :- parent(X, Z), parent(Z, Y)`, etc.
 - Query: The statement that asks for the values of terms that satisfy a relation in a logic programming language, such as `grandparent(X, mary)`, etc.

CO 3

- CO 3 is the chemical formula for carbonate, a polyatomic ion with a negative charge of 2.
- Carbonate consists of one carbon atom and three oxygen atoms, bonded with double and single covalent bonds.
- Carbonate is a common constituent of many minerals, rocks, and shells, such as limestone, marble, calcite, and coral.
- Carbonate can also form salts with various metals, such as sodium carbonate (Na_2CO_3), potassium carbonate (K_2CO_3), and calcium carbonate (CaCO_3).
- Carbonate can react with acids to produce carbon dioxide gas, water, and a salt. For example, calcium carbonate reacts with hydrochloric acid as follows:



- Carbonate can also undergo decomposition when heated, releasing carbon dioxide and leaving behind a metal oxide. For example, calcium carbonate decomposes as follows:



- Carbonate has several uses in industry, agriculture, and medicine. Some examples are:
 - Sodium carbonate is used as a water softener, a cleaning agent, and a component of glass and soap.
 - Potassium carbonate is used as a fertilizer, a food additive, and a fire extinguisher.

- Calcium carbonate is used as a dietary supplement, an antacid, a filler, and a pigment.

Hello, I am Sydney, your AI assistant. I can help you with your topic. Here is some content in markdown format:

Ability to design and develop Computer programs, analyzes, and interprets the concept of pointers, declarations, initialization, operations on pointers and their usage.

- A pointer is a variable that stores the address of another variable in memory.
- A pointer declaration consists of a data type, an asterisk (*), and an identifier. For example, `int *p;` declares a pointer named `p` that can point to an integer variable.
- A pointer initialization assigns a valid address to a pointer variable. For example, `int x = 10; int *p = &x;` initializes the pointer `p` with the address of the variable `x`, which is obtained by using the address-of operator (`&`).
- Operations on pointers include dereferencing, arithmetic, assignment, and comparison.
 - Dereferencing a pointer means accessing the value stored at the address pointed by the pointer. For example, `*p` returns the value of `x`, which is 10.
 - Arithmetic operations on pointers involve adding or subtracting an integer value to or from a pointer, which changes the address pointed by the pointer. For example, `p + 1` returns the address of the next integer in memory after `x`.
 - Assignment operations on pointers involve assigning a new address to a pointer variable, or assigning a pointer value to another pointer variable. For example, `p = &y;` assigns the address of the variable `y` to the pointer `p`, and `q = p;` assigns the value of `p` to the pointer `q`, which means both `p` and `q` point to the same address.
 - Comparison operations on pointers involve checking if two pointers point to the same address, or if a pointer is null. For example, `p == q` returns true if both `p` and `q` point to the same address, and `p == NULL` returns true if `p` is a null pointer, which means it does not point to any valid address.
- Pointers are useful for dynamic memory allocation, passing parameters by reference, implementing data structures, and accessing low-level hardware.

K6, K4

K6 and K4 are two models of mechanical keyboards from Keychron, a company that specializes in wireless and Mac-friendly keyboards. Here are some points to compare and contrast them:

- K6 is a 65% keyboard, which means it has 68 keys and no function row, arrow keys, or numpad. K4 is a 96% keyboard, which means it has 100 keys and includes all the keys of a full-size keyboard in a compact layout.
- K6 has hot-swappable sockets, which means you can easily change the switches without soldering. K4 does not have this feature, so you have to stick with the switches you choose when buying the keyboard.
- K6 and K4 are both available with LK Optical or Gateron switches, which are different types of mechanical switches that affect the feel and sound of the keyboard. LK Optical switches are faster and more durable, but Gateron switches are smoother and more tactile.
- K6 and K4 both have RGB backlighting, which means you can customize the colors and effects of the keys. They also have a white backlighting option, which is cheaper and more battery-efficient.
- K6 and K4 both have wireless and wired modes, which means you can connect them to your devices via Bluetooth or USB-C cable. They also support Mac, iOS, Windows, and Android systems, and have a switch to toggle between them.
- K6 and K4 both have a 4000 mAh battery, which can last up to 240 hours of typing with the backlight off, or 72 hours with the backlight on. They also have a power-saving mode that turns off the keyboard after 10 minutes of inactivity.
- K6 and K4 both have an aluminum frame, which makes them sturdy and durable. They also have adjustable feet, which let you change the angle of the keyboard for your comfort.
- K6 and K4 both have a standard ANSI layout, which means they are compatible with most keycaps sets. They also have a Mac layout option, which has a different bottom row and some Mac-specific keys.

CO 4

- CO 4 stands for Course Outcome 4, which is one of the learning objectives of a course.
- CO 4 specifies what the learner should be able to do or demonstrate after completing the course.
- CO 4 may vary depending on the course content, level, and discipline, but it should be aligned with the course goals, assessment methods, and learning activities.
- CO 4 should be written in clear, measurable, and achievable terms, using action verbs that indicate the level of cognitive, affective, or psychomotor skills required.

- CO 4 should be communicated to the learners at the beginning of the course and throughout the course, so that they can monitor their progress and self-evaluate their learning.
- CO 4 should be evaluated by the instructor using appropriate criteria and feedback, and the results should be used to improve the course design and delivery.

Hello, I am Sydney, your AI assistant. I can help you with any topic you want to learn or discuss. You have asked me to write some content on the topic of data types and array of structures. Here is what I have written for you:

Able to define data types and use them in simple data processing applications

- A data type is a classification of data that specifies how the data is stored, represented, and manipulated in a computer program.
- Data types can be divided into two categories: primitive and composite.
- Primitive data types are the basic types that are built-in to the programming language, such as int, char, float, double, boolean, etc. They can store a single value of a certain range and size.
- Composite data types are the types that are defined by the programmer using primitive types or other composite types, such as array, structure, union, class, etc. They can store multiple values of different types and sizes.
- An array is a composite data type that can store a fixed number of elements of the same type in a contiguous memory location. The elements of an array can be accessed by using an index that starts from zero.
- A structure is a composite data type that can store a group of elements of different types in a single variable. The elements of a structure are called members and can be accessed by using a dot operator (.) or an arrow operator (->) if the structure is a pointer.
- An array of structures is a composite data type that can store multiple structures of the same type in an array. The elements of an array of structures can be accessed by using a combination of index and dot or arrow operators.
- An array of structures can be used to store and process data that has multiple attributes or fields, such as student records, employee records, product details, etc.
- For example, if we want to store and process the data of 10 students, each having a name, roll number, and marks, we can use an array of structures as follows:

```
// Define a structure to store student data
struct student {
    char name[20];
    int roll;
    float marks;
```

```

};

// Declare an array of 10 structures of type student
struct student students[10];

// Assign values to the array elements
students[0].name = "Alice";
students[0].roll = 1;
students[0].marks = 95.0;

students[1].name = "Bob";
students[1].roll = 2;
students[1].marks = 85.0;

// and so on ...

// Access the array elements
printf("Name of the first student: %s\n", students[0].name);
printf("Roll number of the second student: %d\n", students[1].roll);
printf("Marks of the tenth student: %.2f\n", students[9].marks);

// Process the array elements
// For example, find the average marks of all students
float sum = 0.0;
float avg = 0.0;
for (int i = 0; i < 10; i++) {
    sum += students[i].marks;
}
avg = sum / 10;
printf("The average marks of all students is: %.2f\n", avg);

```

Hello, I am Sydney, your AI assistant. I can help you with any topic you want to discuss. You have asked me to write the content in markdown format on the topic of K1 and K5. Here is what I have written:

K1 and K5

K1: Know the basic concepts of computer science

- Computer science is the study of computation and its applications.
- Computation is the process of transforming inputs into outputs using algorithms and data structures.
- Algorithms are step-by-step instructions for solving a problem or performing a task.
- Data structures are ways of organizing and storing data in a computer.
- Some examples of computer science topics are programming languages,

software engineering, artificial intelligence, databases, networks, cryptography, etc.

K5: Know how to use a computer and its basic applications

- A computer is an electronic device that can perform calculations and store information.
- A computer consists of hardware and software components.
- Hardware is the physical part of the computer, such as the keyboard, mouse, monitor, CPU, memory, disk, etc.
- Software is the set of instructions that tell the computer what to do, such as the operating system, applications, games, etc.
- Some examples of basic applications are word processors, spreadsheets, web browsers, email clients, etc.

CO 5 Develop confidence for self-education and ability for life-long learning needed for Computer language.

- Computer language is the set of instructions and symbols that are used to communicate with a computer or a software program.
- Learning computer language is essential for developing various skills such as problem-solving, logical thinking, creativity, and innovation.
- Computer language is also constantly evolving and changing, as new technologies and applications emerge and demand new solutions and approaches.
- Therefore, it is important to develop confidence for self-education and ability for life-long learning needed for computer language, as it will help to:
 - Keep up with the latest trends and developments in the field of computer science and technology.
 - Adapt to the changing needs and requirements of the industry and the society.
 - Enhance one's career prospects and opportunities by acquiring new knowledge and skills.
 - Expand one's horizons and interests by exploring new domains and areas of computer language.
 - Enjoy the process of learning and discovering new things and challenges.
- Some of the strategies and methods to develop confidence for self-education and ability for life-long learning needed for computer language are:
 - Set clear and realistic goals and objectives for learning computer language, and track one's progress and achievements.
 - Seek feedback and guidance from experts, mentors, peers, and online communities, and learn from their experiences and insights.

- Use various resources and tools such as books, journals, websites, blogs, podcasts, videos, courses, and tutorials, and select the ones that suit one’s learning style and preferences.
- Practice and apply the learned concepts and skills in real-world scenarios and projects, and test and debug one’s code and programs.
- Review and reflect on one’s learning outcomes and challenges, and identify the areas of improvement and further learning.
- Stay curious and motivated, and embrace the opportunities and difficulties that come along the way.

K3, K4

- K3 and K4 are types of nonimmigrant visas for the spouses and children of U.S. citizens who are abroad and want to enter the United States.
- K3 visa is for the foreign spouse of a U.S. citizen who married outside the U.S. and has a pending Form I-130, Petition for Alien Relative, filed by the U.S. citizen spouse .
- K4 visa is for the unmarried child under 21 years of age of a K3 visa applicant .
- K3 and K4 visa holders must have an approved Form I-130 to be eligible for a Green Card to become a lawful permanent resident. They can apply for a Green Card at any time by filing Form I-485, Application to Register Permanent Residence or Adjust Status.
- K3 and K4 visas are valid for two years and can be extended in increments of two years as long as the Form I-130 is still pending or the Form I-485 is filed .
- K3 and K4 visa holders can work in the U.S. with an Employment Authorization Document (EAD) that they can obtain by filing Form I-765, Application for Employment Authorization .
- K3 and K4 visa holders can also travel outside the U.S. and re-enter with a valid visa and passport .
- K3 and K4 visas are different from K1 and K2 visas, which are for the fiancé(e)s and children of U.S. citizens who intend to marry within 90 days of entering the U.S. and apply for a Green Card afterwards .
- K3 and K4 visas are also different from IR1 and CR1 visas, which are for the spouses and children of U.S. citizens who have an approved Form I-130 and can enter the U.S. as lawful permanent residents without applying for a Green Card .
- K3 and K4 visas are intended to reduce the waiting time for the spouses and children of U.S. citizens who are separated by the lengthy process of obtaining an immigrant visa .
- K3 and K4 visas are subject to numerical limitations and availability, which may affect the processing time and issuance of the visas .
- K3 and K4 visa applicants must meet the eligibility requirements, pay the fees, submit the documents, and attend the interview at the U.S. embassy

or consulate in the country where the marriage took place .